scientific reports

Check for updates

OPEN

ISAAF: an IoT security and attack prevention framework using AI-driven predictive analytics

Khaoula Karam[1]✉, Abderrahmane Aqachtoul[1], Abderrahmane Elamrani[1,2], Mehdi Najib[1], Najat Rafalia[3], Idriss Moumen[3], Jaafar Abouchabaka[3] & Mohamed Bakhouya[1]

The Internet of Things (IoT) is reshaping domains, such as healthcare, agriculture, and industry by enabling real-time connectivity among constrained devices. However, the lightweight Message Queuing Telemetry Transport (MQTT) protocol exposes these systems to severe cyber threats, including DoS, Bruteforce, Malformed, Flood, and Slowite attacks. While machine learning (ML) and deep learning (DL) models trained on simulated benchmarks, such as MQTTset, have shown promise, evaluation results indicate that these models achieved high accuracy in controlled environments but failed to generalize to real-world traffic. To address this limitation, MQTTEEB-D was introduced as a novel real-world intrusion dataset collected from an operational IoT testbed. Building on MQTTEEB-D, a layered and AI-driven security framework is introduced for real-time intrusion detection and automated mitigation. For instance, Decision Tree (DT) and Gated Recurrent Unit (GRU) accuracies, using MQTTset, dropped to 8% and 21% when tested on real data. However, after retraining both models on MQTTEEB-D, the results showed a noticeable improvement; DT reached 87% and GRU 86.5% accuracy. The framework was deployed and tested in real-sitting scenarios and experimental results demonstrated efficient attacks' detection and mitigation with near-real-time responsiveness. Overall, the findings confirm that the proposed framework and its related services provide a scalable, deployable, and cross-domain security solution for real-world IoT applications.

Keywords IoT security, MQTT protocol, Cyberattack detection, Machine learning, Deep learning, MQTTEEB-D

The rapid expansion of IoT has transformed sectors, such as healthcare, agriculture, and industrial systems by enabling real-time data exchange among interconnected devices[1]. While these advances deliver immense benefits, they also create new attack surfaces across interconnected devices and communication networks, particularly through lightweight communication protocols, like the MQTT. For instnace, MQTT is widely adopted, as an IoT communication protocol, for its low overhead and suitability for constrained devices, yet it remains highly vulnerable to cyber-attacks, such as Denial of Service (*DoS*), Bruteforce Authentication (*Bruteforce*), Malformed Data (*Malformed*), Slow DoS against Internet of Things Environments (*Slowite*), and MQTT Publish Flood (*Flood*). In contexts, such as healthcare, where both data confidentiality and service availability are critical, these vulnerabilities pose severe risks[2–4]. Traditional security mechanisms, often reactive in nature, struggle to prevent such evolving threats and are typically benchmarked only on simulated datasets, like MQTTset[5]. Although valuable, these datasets fail to capture the variability and noise of real-world IoT traffic, limiting the robustness and deployability of ML/DL-based intrusion detection systems. Recent studies have explored these challenges through advanced approaches, such as transformer-based intrusion detection and federated learning for MQTT and IoT environments[3,6–10], highlighting the growing shift toward scalable and adaptive AI-driven security solutions.

To overcome these limitations, we propose a real-time IoT security framework (ISAAF) that integrates pre-trained ML and DL models for predictive attacks' detection and automated prevention. Specifically, we focus on the two best-performing models identified during benchmarking: DT from the ML family and GRU from the DL family. Both were initially trained on MQTTset, but their deployment in real-sitting scenarios highlighted the need for realistic datasets. In this work, we developed MQTTEEB-D[11], a real-world MQTT intrusion

[1]College of Engineering and Architecture, LERMALab & TICLab, International University of Rabat, Sala Al Jadida 11100, Morocco. [2]Smart Systems Laboratory (SSLab), National School of Computer Science and Systems Analysis (ENSIAS), Mohammed V University in Rabat, Rabat 10000, Morocco. [3]Faculty of Sciences, IT Department, Ibn Tofail University, Kenitra, Morocco. ✉email: khaoula.karam@uir.ac.ma

dataset capturing authentic traffic patterns and attacks' traces within a healthcare IoT testbed. Retraining DT and GRU models on MQTTEEB-D significantly improved robustness, enabling accurate, proactive detection of *DoS*, *Bruteforce*, *Malformed*, *Flood* and *Slowite* attacks. ISAAF's deployment was realized using Node-RED for orchestration and ThingsBoard for monitoring, ensuring seamless integration into IoT infrastructures without disrupting ongoing services. Remaining platform's services continue to operate normally, as mitigation actions are applied only to the affected communication channel. This behavior is supported by the latency and jitter analysis presented in the section "Deployment and Real-Time Resilience". In fact, obtained results demonstrate both ISAAF's scalability by handling growth in devices and traffic as well as its cross-environment applicability in other domains, such as precision agriculture and environment monitoring. It is worth noting that the work conducted in this paper is part of the MGFARM project (*Smart Stand-Alone Microgrids for Agricultural Electrification*), where the main objective is to develop Smart Micro-Grids based on the production of renewable energy in order to support sustainable development of smart and energy-efficient farming systems[12]. The project aims to develop and deploy a blockchain-based IoT platform for secure, decentralized energy sharing and efficient resource management within interconnected microgrids[13].

In summary, this work makes four key contributions:

- **Dataset realism:** construction of MQTTEEB-D, a real-world dataset that addresses the shortcomings of simulated benchmarks by capturing authentic IoT traffic and attack scenarios.
- **Benchmarking and validation:** systematic evaluation of multiple ML and DL models on MQTTset, identification of DT and GRU as best candidates, and confirmation of their robustness through retraining and testing on MQTTEEB-D.
- **Architecture design:** development of ISAAF, a novel layered architecture that integrates predictive modeling, attacks detection, automated response, and monitoring into a unified workflow, explicitly covering data capture, preprocessing, and intrusion detection and protection.
- **Framework integration:** deployment of ISAAF in a real IoT testbed (Node-RED + ThingsBoard) and validate its effectiveness under live operating conditions, demonstrating practical and reliable defense against MQTT-based attacks.

The remainder of this paper is organized as follows. The section *Related Work* reviews existing work from literature. The section *ISAAF Framework* introduces the proposed architecture. The section *Materials and Methods* describes datasets and modeling procedures. The section *Experimental Results* presents performance comparisons of ML/DL algorithms. Finally, the section *Conclusion and Perspectives* outlines key findings while highlighting future directions.

Related work

IoT ecosystems combine heterogeneous devices and protocols, which makes end-to-end security difficult to guarantee at scale. Resource-constrained endpoints limit the use of strong primitives (for example, advanced encryption, frequent patching, and mutual authentication), leaving systems exposed to eavesdropping, tampering, and unauthorized access[22,23]. MQTT, while popular for its lightweight publish–subscribe model, broadens the attack surface to *DoS*, *Bruteforce*, and *Slowite*[24]. Protocol hardening via TLS (Transport Layer Security), session resumption, selective payload encryption, OAuth 2.0, or mTLS improves confidentiality and integrity. These solutions by themselves do not, however, neutralize behaviorally subtle or network-level intrusions and remain insufficient for large-scale heterogeneous IoT environments[25,26]. This limitation has motivated extensive research into AI-driven intrusion detection and response, specifically for MQTT-centric IoT systems.

Recent reviews have emphasized the growing role of AI-driven and hybrid detection techniques in cybersecurity, highlighting both their strengths and limitations in evolving IoT environments[6]. The first wave of work focused on characterizing the vulnerabilities of MQTT. For instance, Soni and Makwana[14]surveyed MQTT weaknesses and operational pitfalls, clarifying protocol risks and motivating analytics-based defenses. However, they did not propose a concrete IDS/IPS architecture nor provide empirical validation, leaving open the challenge of building deployable defenses. Building on this groundwork, Hindy et al[6]. explored deep sequence modeling for MQTT traffic by combining CNN (Convolutional Neural Network) and LSTM (Long Short-Term Memory). Their work showed that temporal modeling improved recognition of evolving patterns in smart-home and IIoT settings, illustrating the promise of DL for MQTT telemetry. The main limitation was that evaluations were performed only on simulated traces without reporting latency or overhead, which made deployment implications unclear.

Alzahrani et al[15]. broadened the methodological scope by benchmarking KNN (K-Nearest Neighbors), LDA (Linear Discriminant Analysis), and CNN–LSTM across multiple MQTT datasets in smart cities, healthcare, and IIoT. They confirmed the superiority of ML/DL over rule-based baselines and highlighted trade-offs between shallow and deep models. However, like earlier work, their results suffered from the well-known generalization gap because most datasets were synthetic or semi-simulated, and the study did not incorporate a concrete mitigation loop. In response to robustness concerns, Siddharthan et al. proposed SENMQTT-SET[16], an ensemble-based IDS with multi-cascade feature selection. This improved resilience and efficiency on simulated MQTT traffic in smart-home scenarios, but the reliance on synthetic traces left unresolved questions about performance under real noise, session churn, and broker-side stress.

Al Hanif and Ilyas[7]turned attention toward constrained devices by designing feature-engineering-centric pipelines that feed AI backends, targeting smart agriculture and IIoT. Their targeted feature selection improved detection accuracy under tight resource constraints, which was a practical advance for embedded devices. The tradeoff was nontrivial preprocessing cost, and the evaluation lacked validation in real-sitting scenarios with attacks' detection and mitigate workflow. Most recently, Alzahrani et al[3]. proposed a lightweight AI framework,

which is optimized for resource limited IoT nodes. Their design balanced detection accuracy and runtime efficiency and acknowledged the compute/memory constraints of IoT devices, but once again, validation relied heavily on synthetic traces and provided only a partial end-to-end view.

Recent 2025 studies have further advanced MQTT intrusion detection toward explainable and ensemble-based paradigms. For instance, Zeghida et al. (2025)[17] introduced the XMID–MQTT framework, integrating explainable ML/DL models to enhance interpretability through SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations), while Solanki and Gupta (2025)[18] proposed an ensemble-learning IDS combining diverse classifiers to improve detection robustness across IoT deployments. These approaches underline the increasing emphasis on transparency and hybridization within modern MQTT security research.

From this in-depth state-of-the-art review,, we have noticed three recurring gaps emerge: (i) many studies depend primarily on simulated datasets that ignore real traffic irregularities, timing jitter, and broker stressors, (ii) complete pipelines that start with capture and preprocessing and end with automated mitigation are rarely demonstrated, and (iii) critical QoS metrics, such as latency, jitter, and bottlenecks are underreported, making real-time viability difficult to assess. Table 1 summarizes representative approaches alongside our ISAAF framework.

Parallel to model and framework development, multiple MQTT intrusion datasets have also been released. MQTTset (2020)[5] became the community baseline, with over 11 million simulated samples covering *DoS*, *Slowite*, *Malformed*, *Bruteforce*, and *Flood*. Its strength is scale and availability, but it is entirely simulated. In the same year, Slowite[19] targeted low-rate DoS scenarios, useful for protocol analysis but too small for models' training. MQTT-IoT-IDS2020[20] extended the landscape by introducing flow-level features and diverse intrusions (DoS, MitM, generic intrusion). This dataset enabled the use of CNN–RNN–LSTM and hybrid models but still lacked real deployment irregularities because it was simulated. SENMQTT-SET (2022)[16] provided 34M records with multi-cascade feature selection, advancing feature engineering but lacking realism. DoS/DDoS-MQTT-IoT (2023)[21] captured live traces in a testbed, adding realism but focusing mainly on flooding attacks. Table 2 synthesizes these datasets together with our developed MQTTEEB-D.

In summary, prior studies advanced MQTT security using ML/DL but suffered from simulated datasets, incomplete pipelines, and limited deployment validation. The dual-dataset strategy adopted here using MQTTset for comparability and MQTTEEB-D for realism directly addresses these issues. Together, Table 1 and Table 2 position ISAAF as a deployment aware framework built on realistic data and motivate the following section, where the framework design is detailed.

ISAAF framework

ISAAF is a multi-layer, closed-loop framework designed for real-time detection and mitigation of cyberattacks in MQTT-based IoT environments. Each layer operates autonomously yet interacts continuously through Legitimate data, mitigation control, and adversarial traffic exchanges. This closed feedback loop ensures continuous adaptation and resilience in dynamic IoT scenarios. For instance, tn the left side of Fig. 1, the *Cyberattack Execution* block models the adversarial domain of the framework. Two sub-boxes *Attackers* and *Attacks* define the threat source and its operational scope. Cyber Attacks (DoS, Slowite, Malformed, Flood, and Bruteforce) are injected directly into the MQTT broker, which also manages Legitimate traffic, guaranteeing realistic coexistence between normal and adversarial flows during experimentation. All attack scenarios were developed and executed under controlled conditions, as detailed in the companion dataset paper[11].

Reference	Methodology	Application	Strengths	Weaknesses
Soni & Makwana (2017)[14]	Survey of MQTT vulnerabilities	General IoT	Clarifies protocol risks; motivates AI-based defenses	No IDS design; no empirical validation
Hindy et al. (2020)[6]	CNN–LSTM IDS	Smart homes, IIoT	Temporal modeling improves attack recognition	Simulated-only evaluation; latency/overhead unreported
Alzahrani et al. (2021)[15]	KNN, LDA, CNN–LSTM	Smart cities, healthcare, IIoT	ML/DL outperform rule-based baselines	Generalization gap persists; no integrated mitigation
Siddharthan et al. (2022)[16]	Ensemble IDS (SENMQTT-SET)	Smart homes, IoT	Robustness via ensembling; efficient feature selection	Synthetic traces; resilience under real noise unclear
Al Hanif & Ilyas (2023)[7]	Feature engineering + AI back-ends	Smart agriculture, IIoT	Accuracy on constrained devices; practical features	Preprocessing cost; limited live-system validation
Alzahrani et al. (2024)[3]	Lightweight AI framework	Resource-limited IoT	Balanced accuracy vs. efficiency; smaller footprint	Validation relies on synthetic data; partial end-to-end view
Zeghida et al. (2025)[17]	XAI-oriented ML/DL IDS (XMID–MQTT)	MQTT in IoT	Improves interpretability (SHAP/LIME) and feature transparency	Increased model complexity; limited edge-device validation
Solanki & Gupta (2025)[18]	Ensemble IDS (GNB, K-SVM, MLP + LR)	MQTT-based IoT Applications	Combines multiple learners for higher detection accuracy; scalable to diverse IoT domains	Requires tuning; limited cross-domain evaluation
ISAAF (this work)	**DT + GRU retrained on MQTTEEB-D in a layered, deployment-aware pipeline with automated mitigation**	**Healthcare IoT extensible to other domains (e.g., agriculture, IIoT)**	**Validated on real testbed data; dual-dataset strategy; near-real-time detection with orchestration and monitoring**	**Authors' note: Future improvements include edge optimizations, acceleration, explainability, and broader cross-domain tests.**

Table 1. Representative AI-driven approaches for MQTT security (2017–2025) and the proposed ISAAF framework.

Dataset	Year	Size (Samples)	Attacks covered	Environment	Key features	Methods used in prior works
MQTTset[5]	2020	11,915,716	*Legitimate, DoS, Slowite, Malformed, Bruteforce, Flood*	Simulated IoT Network	ML-ready dataset, packet capture	NN, RF, NB, DT, GB, KNN
Slowite[19]	2020	∼9,202	Low-rate *DoS*	Simulated IoT Network	Exploits MQTT protocol weaknesses	– (attack dataset, no ML baselines)
MQTT-IoT-IDS2020[20]	2021	N/A	*DoS*, MitM, Intrusion	Simulated IoT Network	Bi-flow and packet-flow features for IDS	LR, KNN, DT, RF, SVM, NB; later DL: DNN, CNN–RNN–LSTM, LSTM
SENMQTT-SET[16]	2022	∼34,800,000	*DoS*, Broker & Subscriber Attacks	Simulated IoT Network	Multi-cascade feature selection	LR, KNN, SVM, NB, DT, RF, GB
DoS/DDoS-MQTT-IoT[21]	2023	424,716	*DoS*, DDoS	Real IoT Testbed	Physical deployment, diverse DoS/DDoS patterns	– (dataset paper, no ML baselines)
MQTTEEB-D[11]	**2025**	**222,813**	***Legitimate, DoS, Slowite, Malformed, Bruteforce, Flood***	**Real IoT Testbed**	**Precise attack timestamps, multi-protocol monitoring, live execution**	**DT, RF, GB, XGB, LightGBM, AdaB, CatBoost, KNN, SVM, LDA, NB, MLP, GRU, LSTM, RNN**

Table 2. Comparative view of MQTT intrusion datasets (2020–2025).

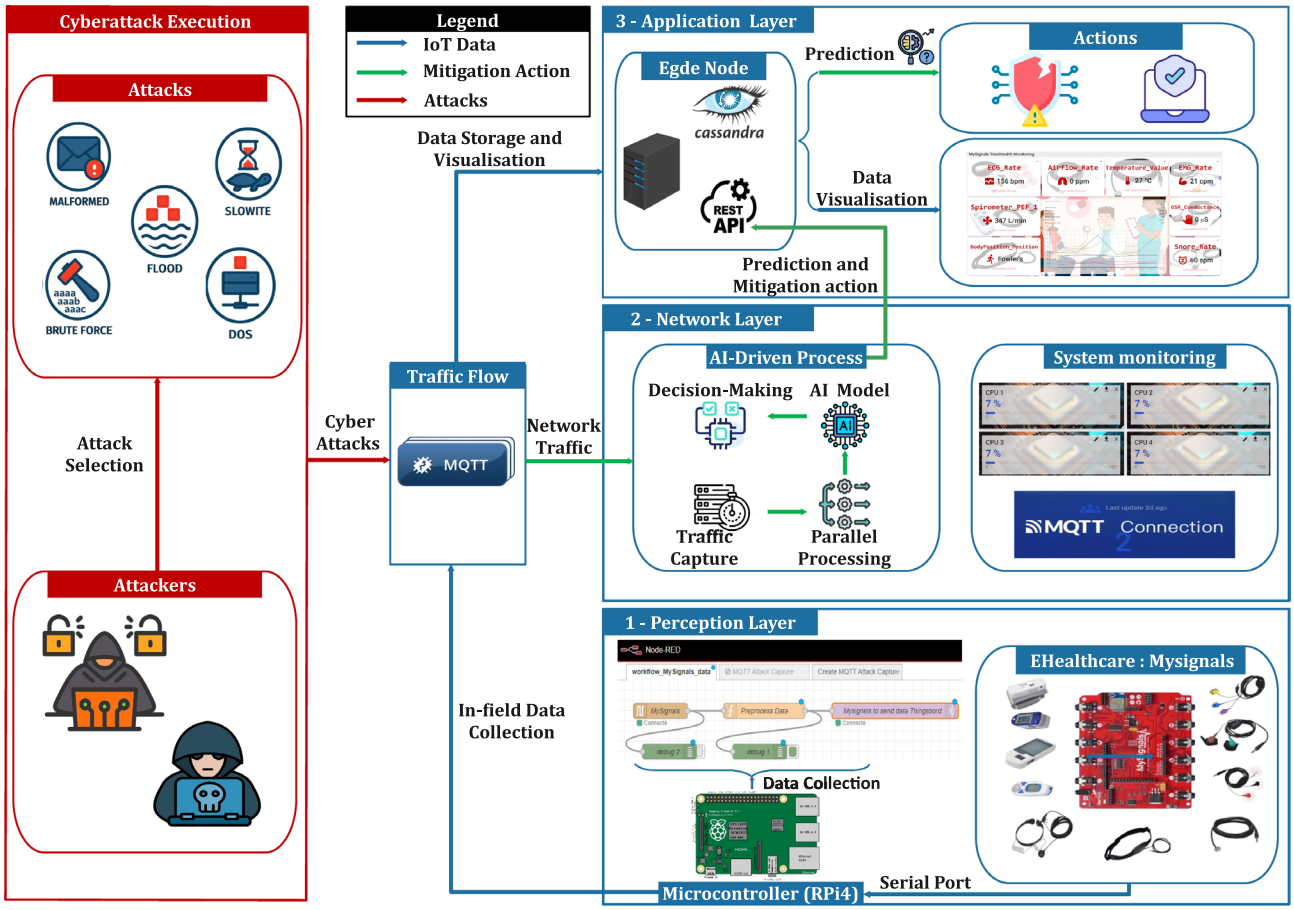


Fig. 1. Layered ISAAF architecture used in our deployment, showing end-to-end data capture, AI-driven analysis, prevention, and monitoring.

{Eight structured phases: data collection, preprocessing, AI analysis, attacks introduction, prediction, prevention, and administrator notification.}

Start
Phase 1: Data Collection
Collect data from sources
if data is complete **then**
    Proceed to network connection
else
    Loop back to data collection
end if
Phase 2: Network Connection
Connect to the network
if connection established **then**
    Proceed to data processing and AI analysis
else
    Loop back to network connection
end if
Phase 3: Data Processing and AI Analysis
Process data and perform AI analysis
if processing successful **then**
    Proceed to application use
else
    Loop back to data processing
end if
Phase 4: Application Use
Utilize processed data in the application
if ready **then**
    Proceed to simulate cyberattacks
else
    Loop back to application use
end if
Phase 5: Simulate Cyberattacks for Testing
Simulate cyberattacks on the system
Phase 6: Attack Detection
if attack detected **then**
    Proceed to prevention
else
    End simulation
end if
Phase 7: Prevention
Implement prevention action
if success **then**
    Proceed to notification
else
    Loop back to prevention
end if
Phase 8: Notification
Send a notification to the administrator
End

Algorithm 1. ISAAF: Real-Time Data Collection and Attack Simulation Workflow

The *Perception Layer* handles in-field data streams and serves as the primary sensing and data-acquisition component of the proposed framework, ensuring continuous and reliable input from IoT devices. For instance, in our healthcare testbed, a *MySignals* board aggregates, via serial communication interfaces (i.e., serial port), a wide range of physiological multi-sensor signals, such as ECG, airflow, body temperature, EMG. A *Raspberry Pi 4* hosts the *Node-RED* runtime, executing data stream flows by performing light filtering, normalization, and packet formatting before MQTT publication. It is worth noting that this design is domain-agnostic, replacing the biomedical sensors with agricultural or industrial ones requires no architectural change, thereby illustrating ISAAF's modularity and adaptability.

At the core, the *Network Layer* acts as the data-processing and decision-making center. As depicted in Fig. 1, incoming streams first enter the *AI-Driven Process* module and processed through four sequential components: *Traffic Capture*, *Parallel Processing*, *AI Model*, and *Decision-Making*. *Traffic Capture* parses MQTT frames to extract discriminative protocol-level features; *Parallel Processing* distributes data batches across threads to minimize latency; the *AI Model* executes the deployed classifier (DT or GRU); and *Decision-Making* interprets predictions to generate prevention commands. In parallel, a *System Monitoring* module visualizes CPU load, memory use, and MQTT connectivity to ensure broker stability during stress. For instance, an average end-to-end cycle of 10–11 s (sensor-to-publication) was measured, including less than 4 s for inference and 1–3 s for mitigation values confirming nearly real-time responsiveness under operational conditions.

The *Application Layer* closes the loop by combining intelligent mitigation, visualization, and data storage. In fact,the *Edge Mode* integrates a *Cassandra* database for scalable time-series management and a *REST API* for automated mitigation actions. When an anomaly is detected, a REST call is triggered and instructs the broker's firewall to block the malicious IP and the port 1883 connection, while Legitimate MQTT traffic continues uninterrupted. In parallel, processed information are forwarded to the dashboard for operator insight. The *Actions* panel confirms successful enforcement and restoration of stable operation. Real-time parameters, such as *ECG_Rate*, *Airflow_Rate*, *Temperature_Value*, and *EMG_Rate* are continuously displayed through the ThingsBoard interface, ensuring observability and auditability.

This organized structure visually captures ISAAF's cyclic sensing–decision–action workflow. More formally, Algorithm 1 details this operational cycle. It encapsulates the eight key phases data collection, network connection, AI analysis, application use, attack simulation, detection, prevention, and administrator notification implemented as an autonomous loop. Each algorithmic step corresponds directly to a visual component in Fig. 1, ensuring one-to-one traceability between conceptual design and real deployment. This integration reinforces the reproducibility and transparency of the proposed architecture.

In summary, ISAAF delivers an interpretable, self-contained, and experimentally validated framework that unites perception, network, and application intelligence into a real-time adaptive defense loop. Its layered

structure, explicit data and control pathways, and verified timing performance confirm its robustness and readiness for large-scale IoT cybersecurity applications.

Materials and methods

Data description and generation process

To rigorously evaluate the ISAAF framework for attack detection in MQTT-based IoT environments, two complementary datasets were employed: the benchmark *MQTTset*[5] and our *MQTTEEB-D* dataset[11]. MQTTset ensures comparability with prior work, while MQTTEEB-D provides realistic traces from a physical testbed. Together, they form a dual foundation for standardized benchmarking and validation in real-sitting scenarios.

The MQTTset is a large-scale dataset, publicly available on Kaggle 1. It was mainly designed to emulate MQTT communications with both benign and adversarial traffic. MQTTset contains approximately 11.9 million records (∼1.6 GB) spanning *Legitimate* traffic and five attack categories: *DoS*, *Slowite*, *Bruteforce*, *Malformed*, and *Flood*. The dataset is distributed in several variants, including a reduced file (`mqttdataset_reduced.csv`) and pre-split train/test sets. We used the reduced version with an 80/20 split to balance computational feasibility and statistical robustness. Features span TCP and MQTT layers (e.g., flags, inter-segment timing, connection flags, QoS indicators, and packet identifiers). Table 3 provides a condensed summary of the **34 original features**, where grouped entries (for example, `mqtt.conflag.*` and `mqtt.sub.*`) represent related MQTT subfields, which are aggregated for readability. Full field names and definitions remain available in the public dataset documentation for reference.

To overcome the limitations of synthetic data and mirror operational conditions, we constructed MQTTEEB-D ([11], 1), in a healthcare-oriented testbed with MySignals sensors, a Raspberry Pi 4 device, and a ThingsBoard MQTT broker. *Legitimate* data streams were continuously published, while adversarial scenarios targeted the broker with the same five attack types present in MQTTset. Traces were captured with multi-threaded PyShark and synchronized with metadata to ensure precise labeling. The final dataset contains 222,813 records with 13 features and a six-class target. Figure 2 illustrates the overall testbed setup, including sensors, broker, and monitoring infrastructure, where MySignals sensors emulate healthcare IoT devices, a Raspberry Pi 4 acts as the edge node performing local data acquisition and attack simulation, and the ThingsBoard broker serves as the central communication hub. This description clarifies the role of each hardware element and the physical data streams. While Fig. 3 details the workflow from data acquisition through attack execution, capture, and labeling. The features included in MQTTEEB-D are summarized in Table 4, covering TCP-level attributes, MQTT message indicators, and protocol metadata, thereby providing a structured overview of how each captured parameter reflects real operational conditions during attack execution. This concise synthesis helps readers quickly identify the relevance of each feature and understand how the dataset bridges raw network behavior with higher-level intrusion semantics. This schema also ensures that both normal operations and anomalous behaviors are encoded with sufficient granularity in order to support intrusion detection research. Beyond traffic features, MQTTEEB-D includes monitoring of broker resource usage during attack execution, enabling validation of operational impact. Detailed analysis of these system-level measurements (CPU, memory, and network load) is presented later in Section 1, where they are discussed in the context of the deployment experiments and real-time resilience evaluation. This reference ensures continuity between the dataset description and the experimental validation, clarifying how infrastructure performance metrics complement network-level intrusion data without duplicating figures or explanations across sections.

It is worth noting that, across both datasets, class labels are encoded consistently as follows: *Legitimate* (0), *DoS* (1), *Slowite* (2), *Malformed* (3), *Bruteforce* (4), and *Flood* (5). This harmonization avoids redundancy, ensures consistency, and provides a single reference scheme for all subsequent figures and results. In summary, MQTTset offers a large-scale, simulated benchmark for reproducible comparison, whereas MQTTEEB-D contributes with authentic packet-level traces and system-level validation from a real IoT environment. Their combined use enables both methodological comparability and deployment-oriented evaluation, ensuring ISAAF is tested across the spectrum from synthetic to realistic IoT traffic.

S no	Name	Description	Protocol
1–3	`tcp.*`	TCP layer attributes: flags, time delta, and header length	TCP
4–7	`mqtt.conack.*`	CONNECT acknowledgment fields (`flags`, `val`, `reserved`, `sp`) defining session status	MQTT
8–14	`mqtt.conflag.*`	CONNECT control flags (`cleansess`, `passwd`, `qos`, `retain`, `uname`, `willflag`, etc.)	MQTT
15	`mqtt.conflags`	Connection flags summary	MQTT
16–20	`mqtt.*`	General MQTT message properties (`dupflag`, `hdrflags`, `kalive`, `len`, `msg`)	MQTT
21–22	`mqtt.msgid/msgtype`	Message identifiers and message types	MQTT
23–24	`mqtt.proto_len/protoname`	Protocol metadata (name and length)	MQTT
25–26	`mqtt.qos/retain`	QoS and retain flags controlling delivery and persistence	MQTT
27–28	`mqtt.sub.*/suback.*`	Subscription and acknowledgment QoS parameters	MQTT
29–33	`mqtt.ver/will*`	Version number and Will message parameters (message, topic, lengths)	MQTT
34	`target`	Class label (Legitimate/Attack category)	Label

Table 3. MQTTset condensed feature schema (**34 total features**). Grouped entries marked with an asterisk (*) represent related MQTT subfields for compactness. Number ranges in parentheses correspond to original feature indices.

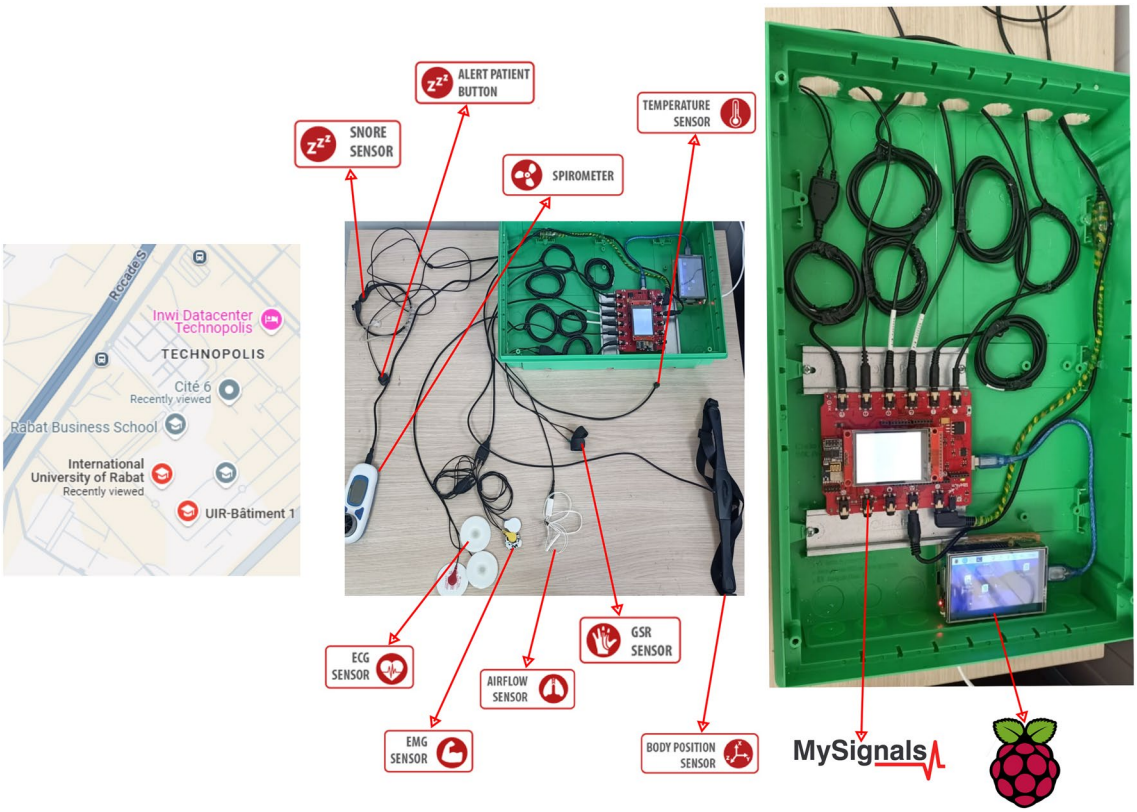


Fig. 2. MQTTEEB testbed setup integrating MySignals sensors, a Raspberry Pi edge device, and a ThingsBoard MQTT broker[11].

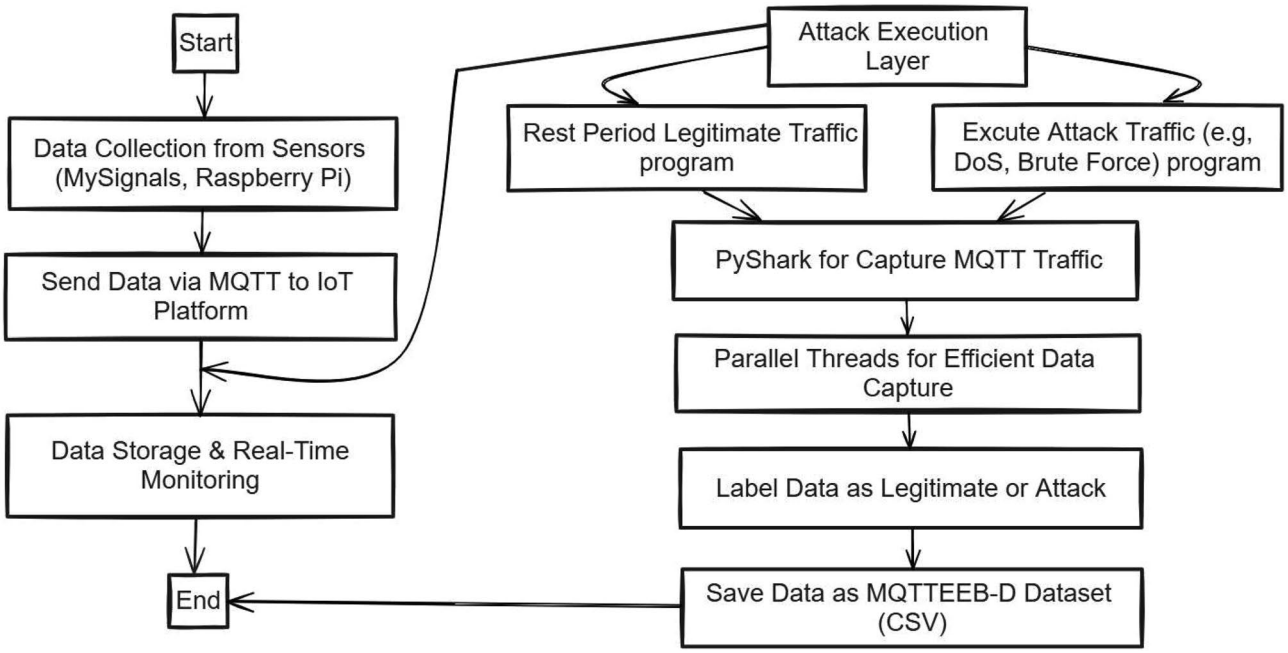


Fig. 3. Data generation workflow for MQTTEEB-D[11]: acquisition, attack execution, packet capture, and labeling.

No	Name	Description	Protocol layer	Data type
1	timestamp	The time when the packet was captured	TCP	object
2	tcp_flags	TCP flags indicating the status of the packet	TCP	int64
3	tcp_time_delta	The time difference between two successive TCP packets	TCP	object
4	tcp_len	The length of the TCP segment	TCP	int64
5	mqtt_conack_flags	MQTT connection acknowledgment flags	MQTT	int64
6	mqtt_conflag_cleansess	Clean session flag in MQTT connection setup	MQTT	int64
7	mqtt_conflags	MQTT control flags, indicating the type of MQTT message	MQTT	int64
8	mqtt_dupflag	MQTT duplicate flag, indicating a re-transmitted message	MQTT	int64
9	mqtt_hdrflags	MQTT header flags, used for control purposes	MQTT	int64
10	mqtt_kalive	MQTT Keep Alive value, indicating the interval between client pings	MQTT	int64
11	mqtt_msg	The actual MQTT message content	MQTT	object
12	mqtt_qos	MQTT Quality of Service level	MQTT	int64
13	target	The target label indicating whether the traffic is normal or under attack	Classification	float64

Table 4. Dataset Log Information of MQTTEEB-D[11].

Dataset preprocessing

To ensure the reliability of subsequent ML and DL modeling, both MQTTset and MQTTEEB-D were preprocessed using a unified pipeline designed to guarantee data quality, comparability, and reproducibility. This pipeline, which forms the data preparation layer of the ISAAF framework, addressed missing values, outliers, heterogeneous feature scales, class imbalance, and categorical encodings, while being guided by a systematic Exploratory Data Analysis (EDA) process. In fact, by standardizing these steps across both simulated and real-world datasets, ISAAF ensured consistent inputs for the downstream intrusion detection modules.

Missing values and outliers: records containing missing or corrupted entries were either imputed or removed, depending on the proportion of affected samples. Outliers were identified and treated using the Interquartile Range (IQR) method, where values outside the bounds defined in Equation 1 were capped to threshold values:

$$Q_1 - 1.5 \times \text{IQR} \;\leq\; x \;\leq\; Q_3 + 1.5 \times \text{IQR}, \tag{1}$$

where Q_1 and Q_3 denote the first and third quartiles (25th and 75th percentiles) of the feature distribution, and $\text{IQR} = Q_3 - Q_1$ represents their interquartile range. This formulation ensures that extreme values outside these thresholds are adjusted without altering the overall distributional shape, thereby minimizing skewness that could bias subsequent learning stages integrated into ISAAF.

Feature scaling: because the datasets integrate heterogeneous attributes spanning TCP-level and MQTT-level features, scaling was necessary to align value ranges and stabilize model convergence. Two techniques were considered: *MinMax* normalization (Equation 2) and *z-score* standardization (Equation 3). Normalization maps attributes into a bounded interval:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad x' \in [0, 1], \tag{2}$$

ensuring that all features contribute proportionally to distance-based learners. In this formulation, x denotes the original feature value, $x_{\min}$ and $x_{\max}$ represent the minimum and maximum values of the feature within the dataset, and x' is the normalized output within the interval $[0, 1]$.

Standardization transforms attributes to zero mean and unit variance, expressed as:

$$z = \frac{x - \mu}{\sigma}, \tag{3}$$

which is advantageous for models assuming Gaussian-like distributions. Here, x is the raw feature value, μ is the mean of the feature across all samples, and σ is the standard deviation, ensuring that the standardized variable z has zero mean and unit variance. In practice, *MinMax* normalization was adopted as the default to preserve interpretability across features and to align with the requirements of neural networks, while *z-score* transformation was applied selectively during experimentation for sensitivity analysis.

Exploratory Data Analysis: was employed to guide feature selection and reduce redundancy. The process integrated three complementary layers: (i) **univariate analysis**, which inspected individual feature distributions through plotting and descriptive statistics; (ii) **bivariate analysis**, where categorical features were assessed using *chi-squared* statistics and numerical features with logistic regression coefficients to quantify their association with the target label; and (iii) **multivariate analysis**, where correlation matrices were used to detect collinearity and redundant variables. Together, these steps systematically filtered and prioritized features, ensuring that only the most informative subset was retained for modeling.

Encoding and class balancing: categorical attributes, such as protocol flags, were numerically encoded using label and binary schemes as appropriate. To address class imbalance, the Synthetic Minority Oversampling

Model	Best parameters used
DT	`max_depth=20, random_state=42`
LightGBM	`n_estimators=100, learning_rate=0.1, num_leaves=31, random_state=42`
XGB	`n_estimators=100, learning_rate=0.1, max_depth=6, random_state=42`
RF	`n_estimators=100, max_depth=20, random_state=42`
GB	`n_estimators=100, learning_rate=0.1, random_state=42`
CatBoost	`n_estimators=100, learning_rate=0.1, depth=6, random_state=42, verbose=0`
KNN	`n_neighbors=7`
MLP	`hidden_layer_sizes=(100,), max_iter=500, random_state=42`
SVM	`kernel=rbf, C=1, probability=True, random_state=42`
LDA	Default (`LinearDiscriminantAnalysis`)
NB	Default (`GaussianNB`)
AdaB	`n_estimators=100, learning_rate=0.1, random_state=42`

Table 5. Best hyperparameters for each ML model.

Model	Architecture	Optimizer	Loss function	Epochs	Minibatch size
GRU	Input: GRU(100), ReLU; Hidden: 4 layers ([L1: GRU(100), ReLU], [L2: Dense(150), ReLU], [L3: Dense(128), ReLU], [L4: Dropout(0.25)]); Output: Dense(6), Softmax	Adam, LR=0.001	Sparse Categorical Cross-Entropy	100	250
LSTM	Input: LSTM(50), ReLU; Hidden: 4 layers ([L1: LSTM(50), ReLU], [L2: Dense(40), ReLU], [L3: Dense(25), ReLU], [L4: Dropout(0.25)]); Output: Dense(6), Softmax	AdamW, LR=0.001	Sparse Categorical Cross-Entropy	150	300
RNN	Input: SimpleRNN(50), ReLU; Hidden: 4 layers ([L1: SimpleRNN(50), ReLU], [L2: Dense(40), ReLU], [L3: Dense(25), ReLU], [L4: Dropout(0.25)]); Output: Dense(6), Softmax	AdamW, LR=0.001	Sparse Categorical Cross-Entropy	100	250

Table 6. Hyperparameters of DL models.

Technique (SMOTE) was applied. This method generated synthetic samples along nearest-neighbor vectors, ensuring equitable class representation without distorting feature distributions.

In summary, the preprocessing established a harmonized and transparent pipeline for both MQTTset and MQTTEEB-D datasets. This ensured consistency across simulated and real-world environments, improved models' stability, and enabled results reproducibility. As part of the ISAAF framework, this preprocessing layer guarantees that the subsequent detection and prevention modules operate on balanced, normalized, and representative data. The effectiveness of these preprocessing steps and the resulting feature selection are analyzed in detail in the Results section.

ML/DL models training

Building on the preprocessed datasets and engineered features, we framed intrusion detection as a multiclass classification task covering *DoS*, *Slowite*, *Bruteforce*, *Malformed*/*Flood* variants, and *Legitimate* activity. To obtain a comprehensive view across modeling paradigms, we evaluated a diverse set of ML algorithms: DT, Random Forest (RF), Gradient Boosting (GB), Extreme Gradient Boosting (XGB), Light Gradient Boosting Machine (LightGBM), AdaBoost (AdaB), CatBoost, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Linear Discriminant Analysis (LDA), Naïve Bayes (NB), and a Multi-Layer Perceptron (MLP) as a neural baseline. This portfolio spans single-tree learners (DT) for interpretability, bagging ensembles (RF) for variance reduction, boosting families (GB, XGB, LightGBM, AdaB, CatBoost) for strong non-linear decision boundaries, instance-based methods (KNN), kernel-based margin maximizers (SVM), linear discriminants (LDA), and probabilistic generative baselines (NB). The MLP model provides a lightweight neural benchmark for non-linear hierarchies.

To ensure fair and effective comparisons, hyperparameters for each ML algorithm were tuned via grid search, combined with k-fold cross-validation, balancing then predictive accuracy against computational cost. Search ranges were defined per algorithm (e.g., tree depth, number of estimators, learning rate, neighbors, kernel type and C for SVM), with the best configurations selected by validation performance and, in case of ties, by lower runtime. The resulting optimal parameters are reported in Table 5. This tuning process enabled each family to demonstrate its strengths: efficient and explainable baselines (DT, LDA, NB), robust ensembles for high-dimensional and moderately imbalanced data (RF and boosting variants), and a neural reference point (MLP).

Following the ML experiments, we extended the study to DL architectures tailored for temporal IoT traffic analysis. Recurrent neural networks (RNN) were selected for their ability to capture sequential dependencies: GRU, LSTM, and simple RNN. These models were built using structured pipelines, which combined recurrent layers with dense layers, for feature compression, dropout for regularization, and softmax outputs for multiclass classification. Their detailed architectures, optimizers, loss functions, and training hyperparameters are summarized in Table 6.

Training of DL models followed the same rigor as ML: cross-validation with batch processing, careful tuning of epochs and minibatch sizes, and adaptive optimization. Regularization techniques, such as dropout and early stopping, were employed to mitigate overfitting, while learning-rate schedules stabilized convergence. All models

were optimized under the sparse categorical cross-entropy loss to ensure comparability across architectures. Optimization was carried out using the Adam and AdamW optimizers, which apply adaptive moment estimation and decoupled weight decay to improve generalization in resource-constrained environments[27].

Experimental results

Feature analysis and preprocessing

A thorough analysis of MQTT/TCP raw features was conducted to establish a reliable foundation for intrusion detection. At the transport layer, descriptors, such as `tcp.flags`, `tcp.len`, and `tcp.time_delta` captured packet-level behavior, while MQTT fields including `mqtt.conack.flags`, `mqtt.conflag.cleansess`, `mqtt.conflags`, `mqtt.dupflag`, `mqtt.hdrflags`, `mqtt.kalive`, `mqtt.msg`, and `mqtt.qos` provided insights into session state, delivery guarantees, duplicate handling, and keep-alive policies. Together with identifiers, like `mqtt.msgid` and `mqtt.len`, these attributes formed the backbone of the dataset, enabling the discrimination between *Legitimate* traffic and attack scenarios, such as *DoS*, *Slowite*, *Bruteforce*, *Malformed*, and *Flood*.

The initial exploration of raw numerical features revealed heavy-tailed distributions, notably in `tcp.len`, `tcp.time_delta`, and `mqtt.msgid`. Figure 4 illustrates this skewness, where extreme outliers could bias training. Outlier treatment using IQR-based capping mitigated these effects by clipping values beyond statistical bounds, reducing distortions while preserving informative ranges (Fig. 5). Normalization was then applied to scale numerical variables into the [0, 1] interval. As shown in Figs. 6 and 7, features that originally spanned highly unequal ranges became harmonized, ensuring that no single variable disproportionately influenced the model's learning process. This was particularly effective for variables, such as `tcp.time_delta`, `tcp.len`, and `mqtt.msgid`, whose post-normalization profiles demonstrated comparability suitable bahaviour for gradient-based algorithms.

The exploratory data analysis confirmed these adjustments and revealed informative relationships with the target labels. Univariate plots (Figs. 8, 9) showed categorical imbalance: for example, `tcp.flags` was dominated by `0x00000010` and `0x00000018`, while `mqtt.msg` concentrated on a few frequent message types. Numerical features, such as `tcp.time_delta` were clustered near zero with rare long delays, and `mqtt.msgid` exhibited a long tail. Bivariate comparisons with the target classes (Figs. 10 and 11) showed clear separability: `tcp.len` values tended to increase under *DoS* traffic, while `mqtt.msgtype` distinguished *Bruteforce* and *Malformed* scenarios. The correlation matrix (Fig. 12) highlighted structural dependencies, including a perfect inverse relation between `tcp.flags` and `mqtt.hdrflags`, a negative link between `mqtt.`

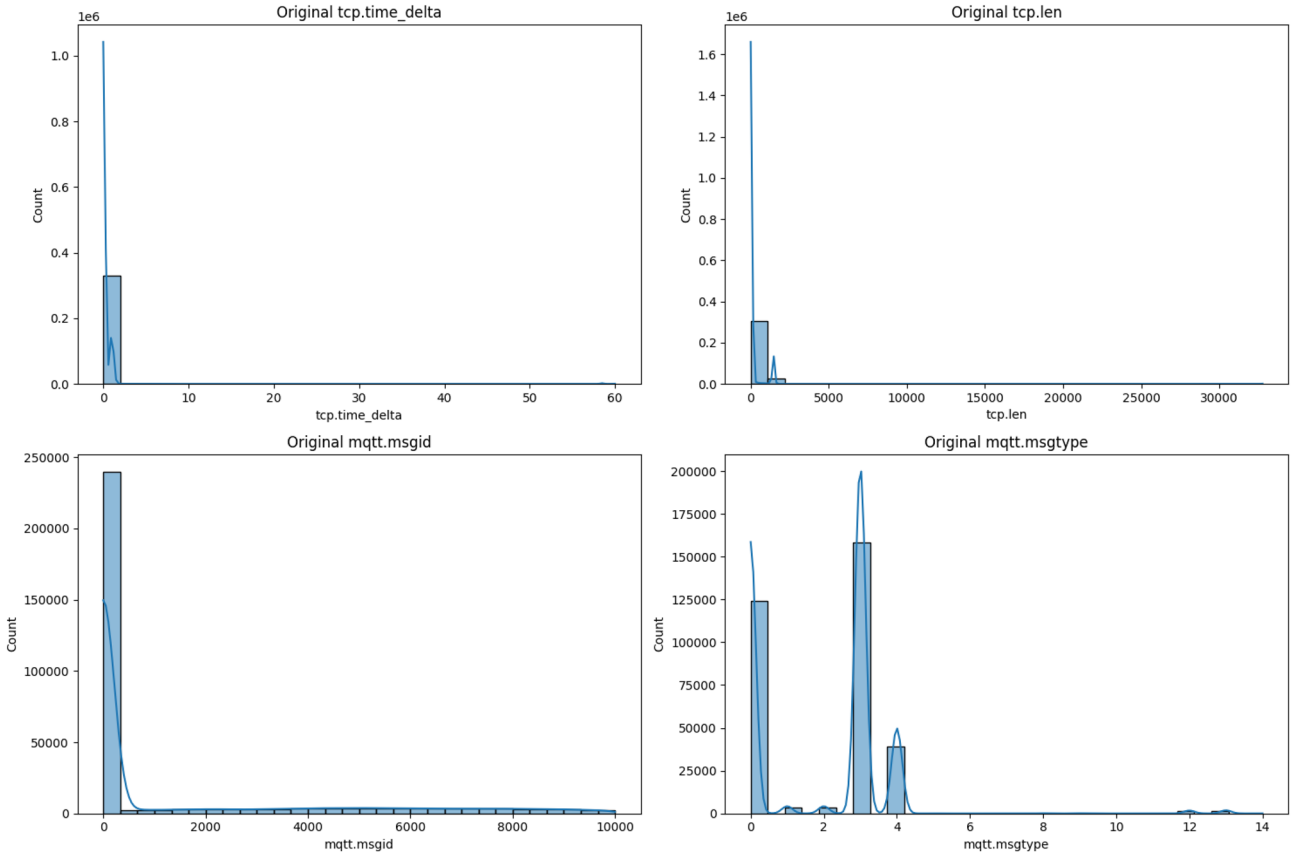


Fig. 4. Original distributions of selected numerical features showing skew and extreme tails.

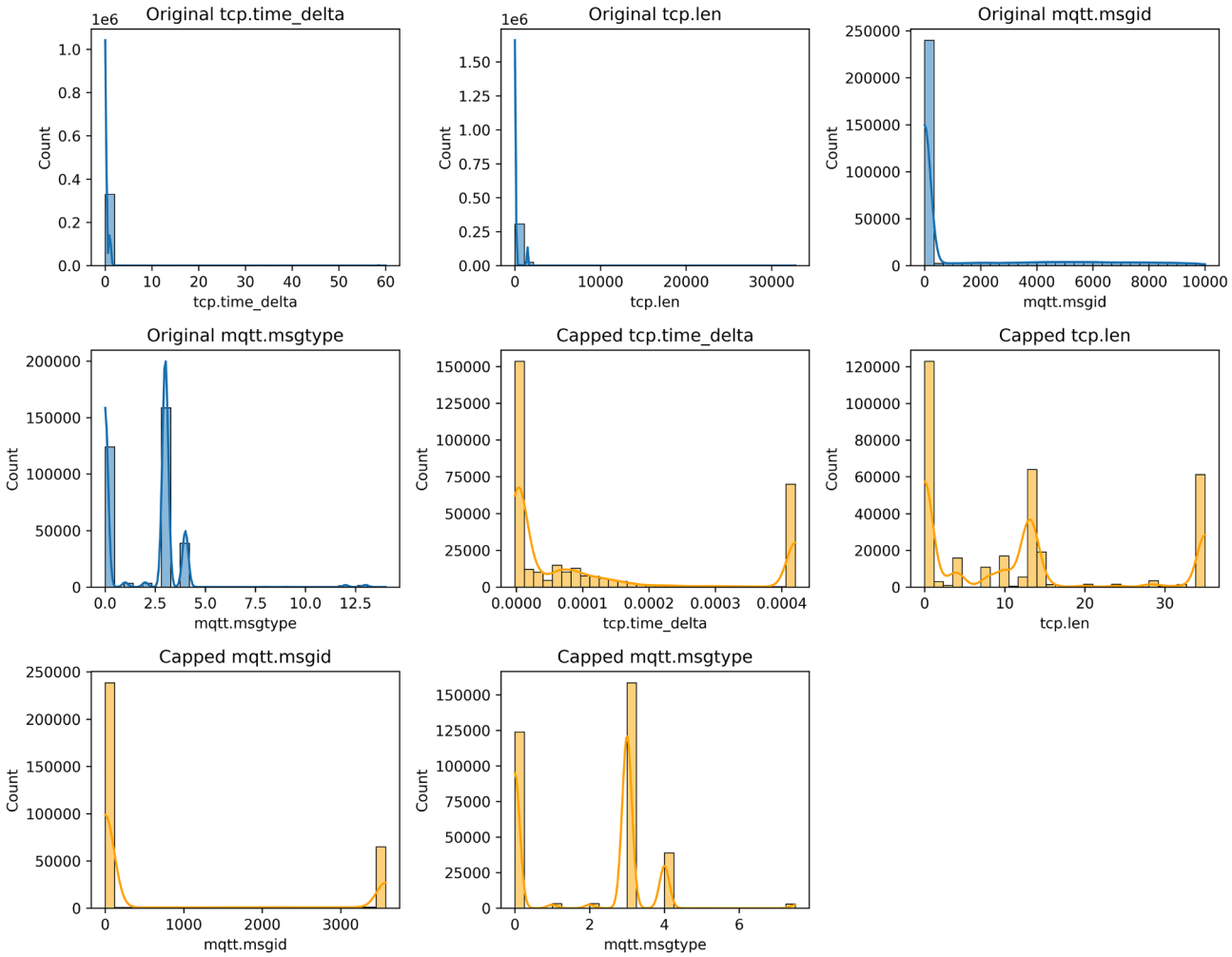


Fig. 5. Distributions after IQR outlier capping, reducing skewness.

`dupflag` and `mqtt.msgtype`, and positive associations between `mqtt.qos` and `mqtt.retain`. These findings informed feature pruning and clarified which attributes carried the strongest predictive signal.

Class imbalance represented another major challenge. *Legitimate* and *DoS* traffic heavily outweighed minority categories, such as *Slowite*, *Bruteforce*, *Malformed*, and *Flood* (Fig. 13A). To avoid bias toward the majority, the synthetic minority oversampling technique was applied, creating synthetic samples for underrepresented classes. The balanced histogram (Fig. 13B) demonstrates the effectiveness of this strategy, ensuring that models trained on the dataset achieve fair recall across all categories rather than collapsing onto dominant classes. SMOTE was applied solely to correct class imbalance and improve minority-class learnability; quantitative error trade-offs are examined later via the confusion-matrix analysis.

Integrating all diagnostics, we consolidated a final feature set that balances discriminative strength with parsimony. Twelve features were retained: `tcp.flags`, `tcp.time_delta`, `tcp.len`, `mqtt.conack.flags`, `mqtt.conflag.cleansess`, `mqtt.conflags`, `mqtt.dupflag`, `mqtt.hdrflags`, `mqtt.kalive`, `mqtt.msg`, `mqtt.qos`, and the `target`. This compact subset reflects the attributes most consistently informative across univariate, bivariate, and multivariate analyses, discarding redundant or weakly discriminative variables while preserving protocol relevance. By focusing on these features, the final schema ensures that ML/DL models are trained on a concise yet information-rich representation of MQTT/TCP traffic.

For the simulated dataset (MQTTset), this pipeline was implemented in a dedicated notebook that details every step from outlier capping to feature selection, ensuring reproducibility and transparency 1. For the real-world dataset (MQTTEEB-D), the same methodology was applied and validated, with results documented in[11]. Importantly, these insights guided the design of our curated dataset: instead of capturing all raw MQTT/TCP fields, we limited collection to the 12 most relevant features identified here and added a timestamp for temporal context. Applying the same analysis confirmed that this reduced schema preserved the strongest discriminative capacity, aligning the simulated and real-world workflows while ensuring high-quality input for the ML/DL models, presented in the next section.

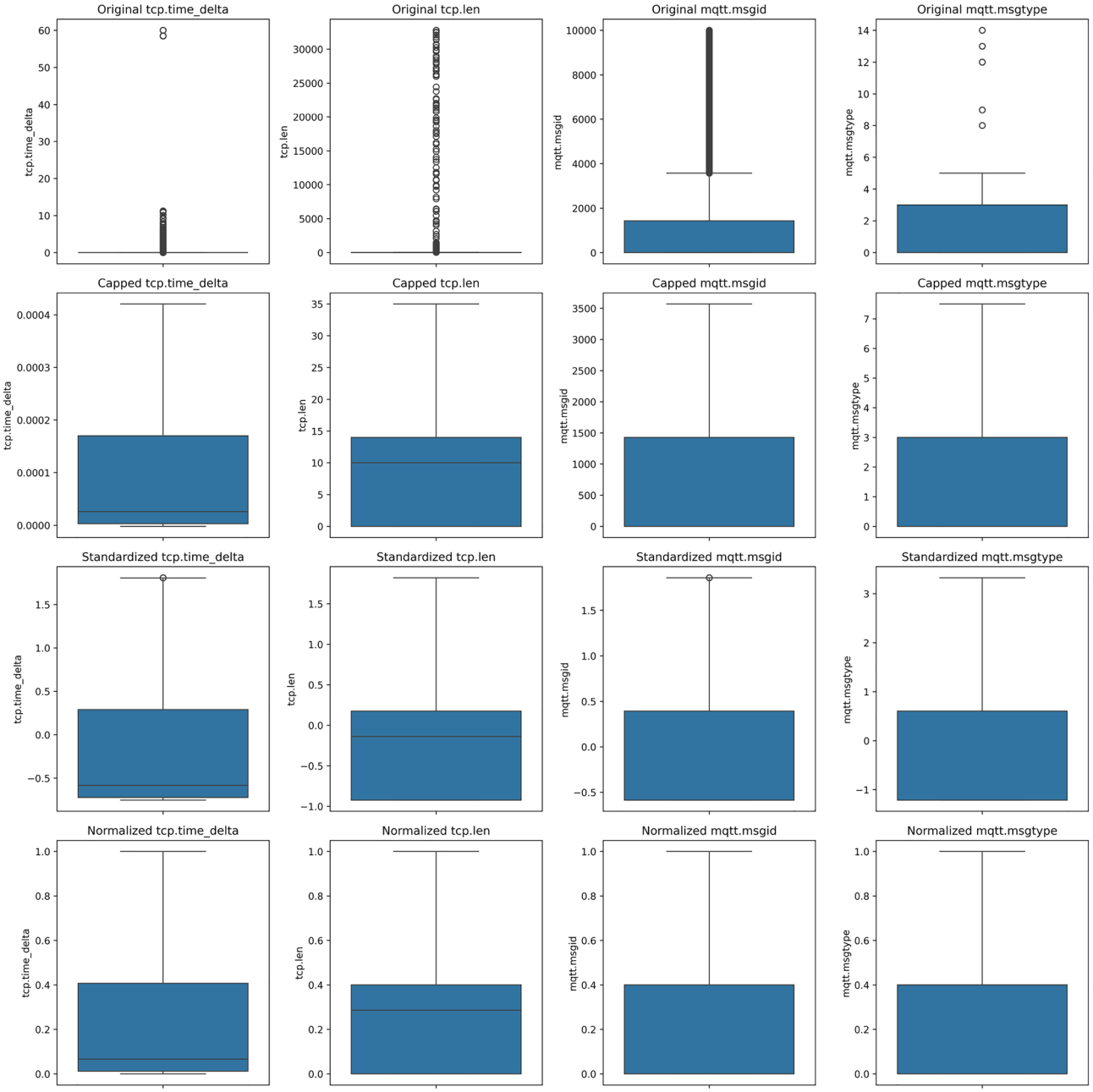


Fig. 6. Box-plots before and after normalization.

Performance evaluation on MQTTset

The comparative study on MQTTset reveals clear separations across methodological families. Tree ensembles (RF, GB, XGB, LightGBM, CatBoost) cluster at the top with ROC AUC values around 0.98 and balanced precision–recall behavior, confirming robustness on high-dimensional IoT traffic. Instance-based KNN and the neural MLP are competitive but consistently trail the ensembles, while linear baselines (LDA, NB) struggle to separate attack from *Legitimate* flows. SVM lands mid-pack, outperforming linear models but not matching boosted trees. Across all candidates, DT offers the most attractive trade-off for practical use: high ROC AUC (0.988 Micro, 0.982 Macro), solid accuracy (0.851), well-balanced precision/recall/F1, and extremely low training (1.202 s) and inference (0.028 s) times–see Table 7.

In addition to these ML results (Table 7), the deep learning (DL) evaluation conducted under identical experimental conditions is summarized in Table 8. This explicit comparison ensures continuity between conventional and sequential architectures, allowing direct benchmarking across methodological families.

Learning-curve diagnostics for DT in Fig. 14A show healthy convergence: as the training set grows, the training score decreases slightly while validation improves, indicating minimal overfitting and stable generalization. ROC analysis, depicted in Fig. 14B, further confirms excellent discrimination, with curves hugging the top-left

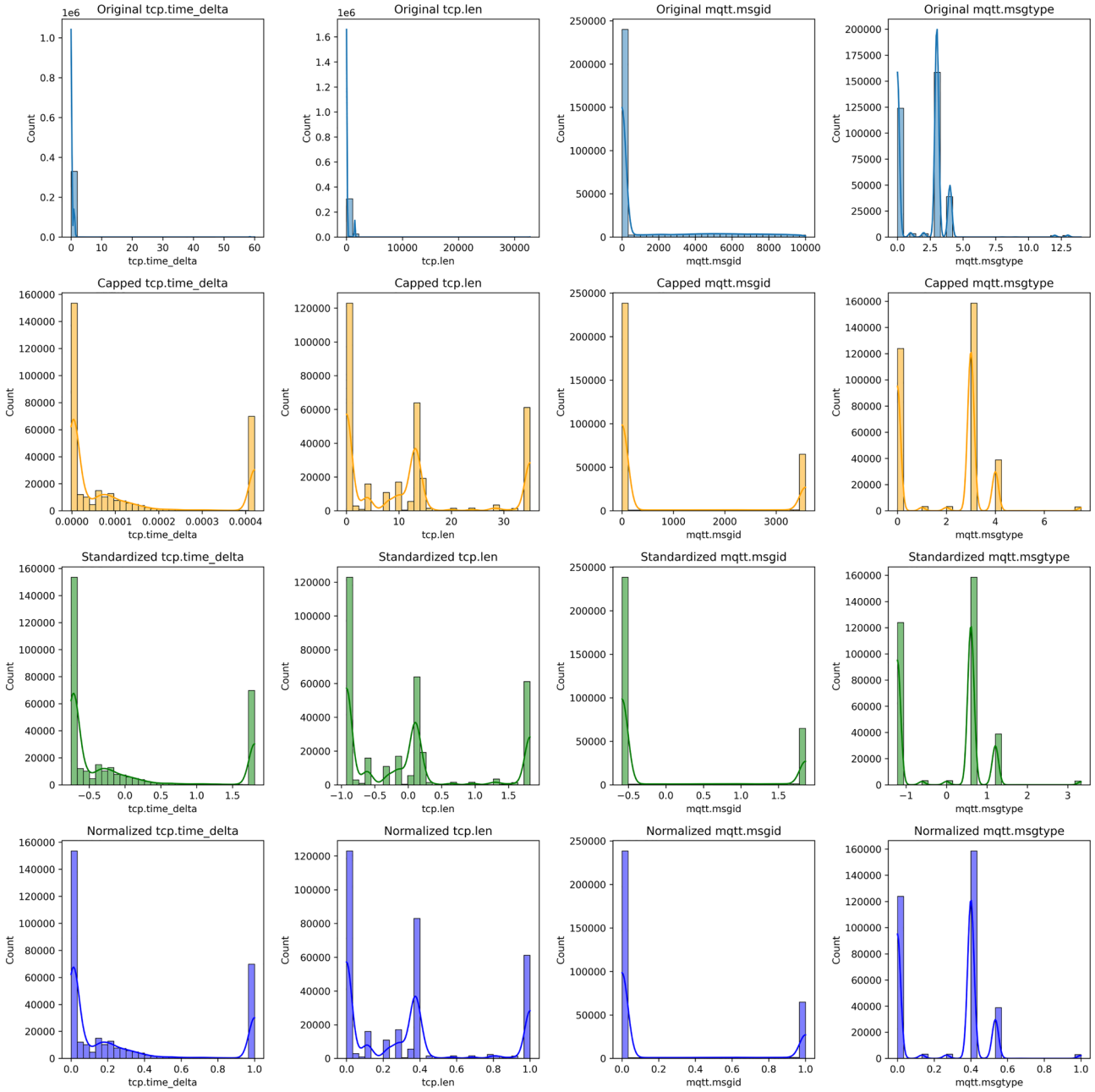


Fig. 7. Distributions after normalization within [0, 1].

corner. Taken together with the low runtime profile, these findings motivate DT as a strong candidate for real-time intrusion detection where interpretability and latency are critical.

Building on the ML evaluation, the DL analysis highlights how sequential models capture temporal dependencies in MQTT traffic. Deep models were therefore examined to characterize recurrent message patterns and contextual transitions. As summarized in Table 8, GRU attained the best overall DL performance (accuracy 84.75%, AUC 0.98), narrowly ahead of LSTM and SimpleRNN, with strong precision–recall balance and acceptable test-time cost. The class-wise ROC results (Fig. 15C) remain uniformly high (AUC ≈ 0.98), and the fold-level learning dynamics (Fig. 15A,B) show fast, stable convergence within the first ten epochs without signs of overfitting–supported by dropout, L_2 regularization ($\lambda = 0.1$), and capacity control. Figures 14 and 15 collectively demonstrate methodological complementarity: DT provides interpretable rule-based partitioning for static features, while GRU excels in temporal abstraction and adaptive sequence modeling. This joint interpretation bridges Tables 7–8, offering a unified view of ML versus DL strengths in the MQTTset environment.

In summary, both DT and GRU emerge as top performers on MQTTset. DT offers the best balance of accuracy (85.1%), interpretability, and runtime efficiency, while GRU provides strong temporal modeling with AUC ≈ 0.98 at modest inference cost. It is worth noting that prior work on MQTTset has reported very high

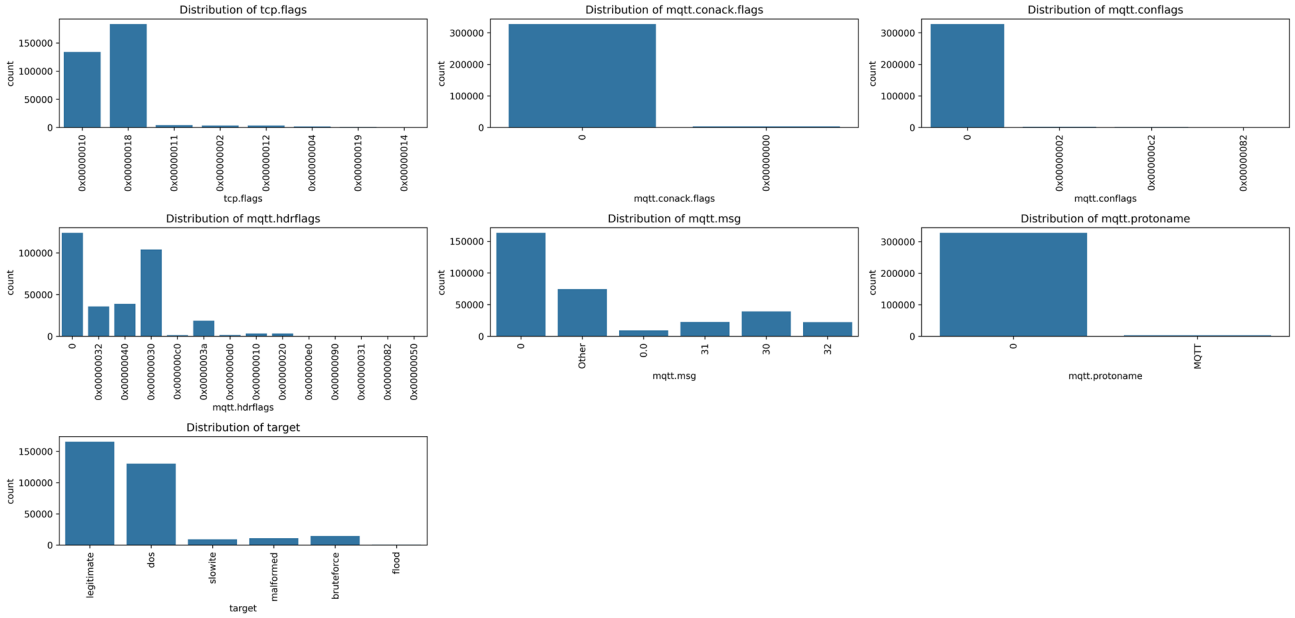


Fig. 8. Univariate categorical feature distributions.

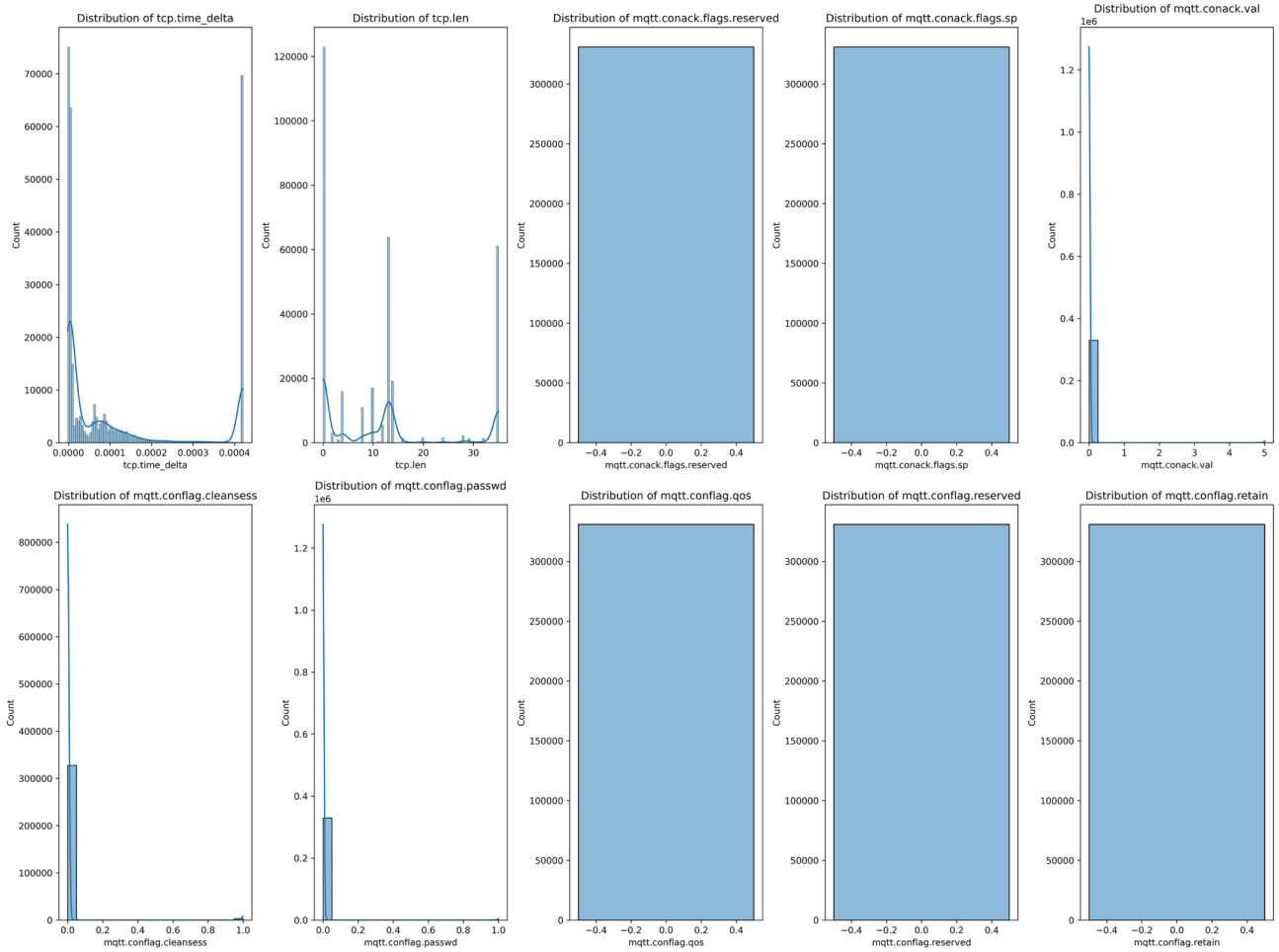


Fig. 9. Univariate numerical feature distributions.

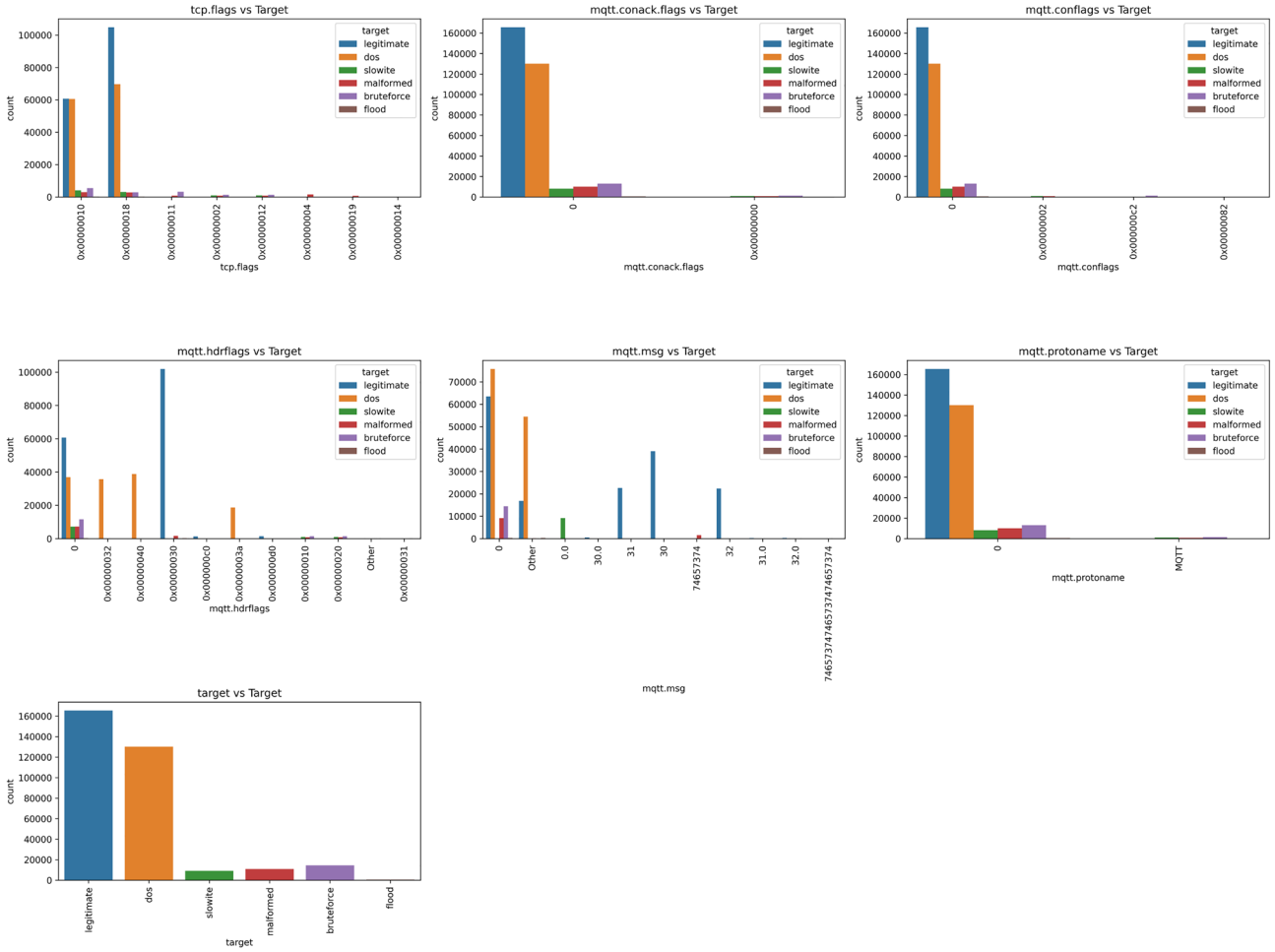


Fig. 10. Categorical features vs. target labels.

accuracies using full feature sets[5], but typically without addressing class imbalance, monitoring generalization, or validating deployability. By contrast, our pipeline constrains learning to a compact, protocol-aware feature set, applies SMOTE, and validates convergence with learning curves. All scripts required to reproduce these ML/DL results are available in the companion repository 1. The next section extends this analysis to the real-world MQTTEEB-D testbed, where the same DT and GRU models are retrained, evaluated, and deployed to assess their operational robustness and readiness for live IoT environments.

Performance evaluation on MQTTEEB-D

We assessed the generalization capability of pre-trained ML/DL models using the MQTTEEB-D dataset, which captures real-world IoT traffic. These models were initially trained on the simulated MQTTset dataset and evaluated on MQTTEEB-D to analyze their adaptability to real-world conditions. As discussed previously, the simplified nature of MQTTset limited the models' ability to represent the complexity and variability inherent in real deployments. Consequently, a retraining phase on MQTTEEB-D was necessary to adapt to the nuanced characteristics of real data, leading to significant improvements across performance metrics.

The quantitative analysis for all confusion matrices in Fig. 16 follows the per-class error-rate formulation defined by Sokolova and Lapalme[28], specifically Equations (6)–(8), which relate true/false positives and negatives to precision, recall, and F1-score. These normalized ratios provide a consistent interpretation of model behavior across all evaluated classes.

The DT model, trained on MQTTset, exhibited a sharp performance drop when tested on MQTTEEB-D. As illustrated in Fig. 16-a, the confusion matrix reveals poor detection of *Bruteforce* and *Flood* attacks, with notable misclassifications across all classes. Quantitatively, more than 60% of *Bruteforce* and nearly 55% of *Flood* samples were incorrectly predicted as *DoS*, while approximately 40% of *Slowite* and *Malformed* instances were falsely labeled as *Legitimate*. This analysis indicates that the DT model primarily captured volumetric patterns rather than protocol-level semantics, resulting in confusion among attacks with similar traffic intensity and duration characteristics.

Similarly, the GRU model, as depicted in Fig. 16-b, displayed partial improvement in distinguishing sequential patterns but still suffered from cross-class leakage. Approximately 45% of *Bruteforce* and 38% of *Flood* samples were still misclassified as *DoS*, whereas 32% of *Slowite* events were misinterpreted as *Legitimate*.

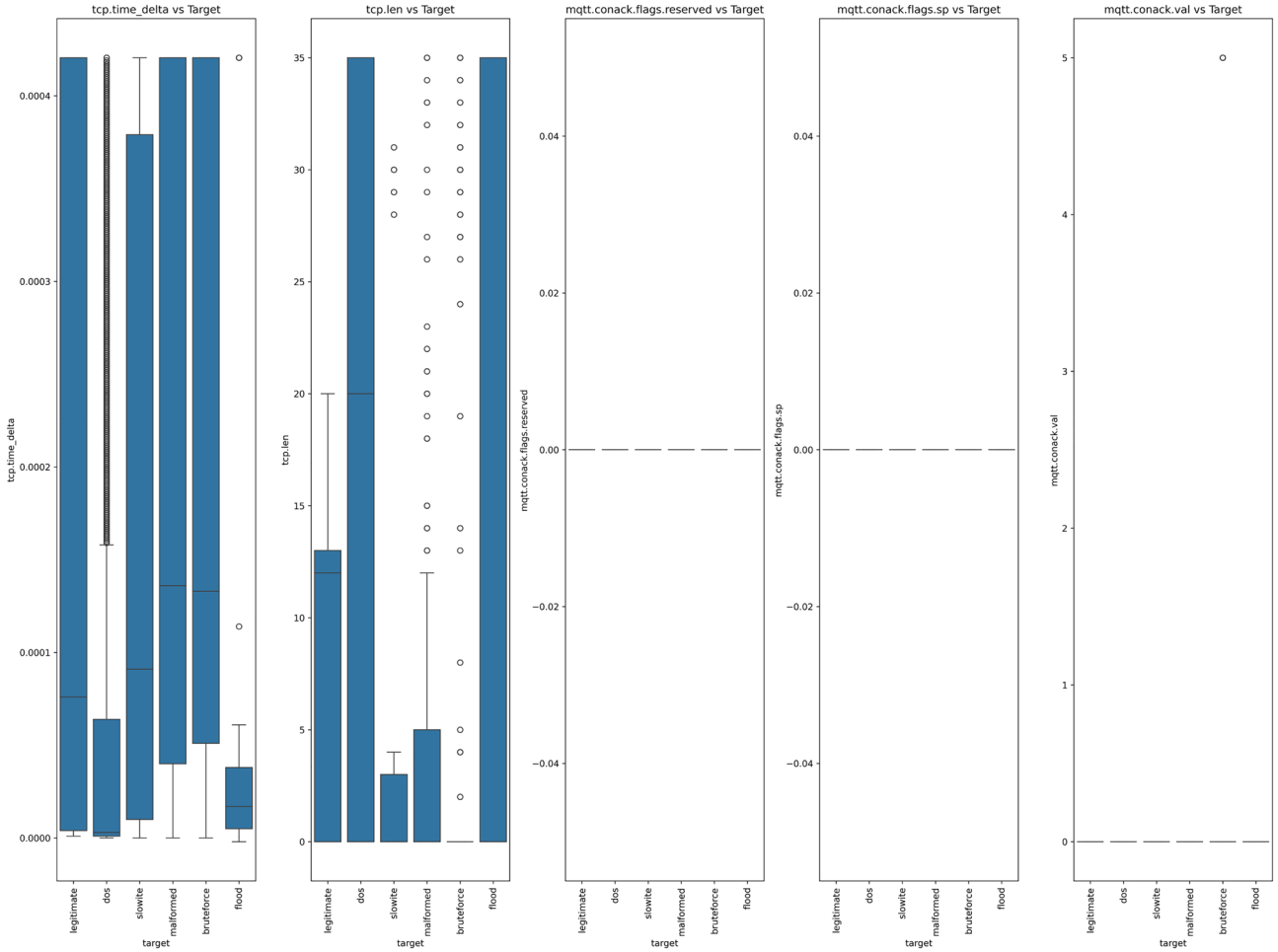


Fig. 11. Numerical features vs. target labels.

Although the GRU captured temporal dependencies better than the DT, its tendency to overfit to recurrent temporal bursts limited its discrimination across low-volume attacks. The improved recall observed for GRU compared to DT confirms its sensitivity to temporal patterns, yet its overall F1-score remained low, highlighting persistent generalization gaps.

Quantitatively, the DT reached only 0.08 in accuracy, with precision, recall, and F1-score also stagnating at 0.08, while the GRU achieved slightly better scores with 0.21 accuracy, 0.29 precision, 0.21 recall, and 0.16 F1-score. These computed values were derived directly from the normalized confusion-matrix counts and follow the same metric definitions, introduced at the beginning of this section, ensuring consistency between both models. These results confirm that pre-training exclusively on MQTTset fails to equip models with the robustness needed for real IoT environments.

Overall, both confusion-matrix analyses (DT and GRU) emphasize that synthetic data can misrepresent real-world temporal and protocol variability, leading to inflated accuracy during simulation and poor generalization upon deployment. Retraining on MQTTEEB-D, which includes genuine timing noise, concurrent connection variability, and mixed attack signatures, was therefore indispensable to achieve reliable model adaptation.

These findings underscore the necessity of training and validating IoT security models on realistic and diverse datasets. During early experimentation, we observed a significant performance drop when pre-trained models were evaluated on real-world traffic, a limitation that highlighted the gap between simulation and actual deployment. It is worth noting that this observation was a key motivation for creating the MQTTEEB-D dataset, designed to capture noisy, heterogeneous, and unpredictable traffic patterns reflective of real-world IoT conditions. By bridging this simulation-to-reality gap, MQTTEEB-D enables the development of more reliable and production-ready security models. Moreover, these empirical results validate the ISAAF framework's ability to perform robust multi-class attack detection using both traditional and DL techniques. However, to evaluate the broader implications of retraining on real-world data and assess model readiness for operational deployment, a more detailed discussion is warranted in the next section.

Deployment and real-world testing

To address the performance issues observed in the pre-trained models, both DT and GRU models were retrained on the MQTTEEB-D dataset. This retraining allowed the models to adapt to the specific characteristics of real-

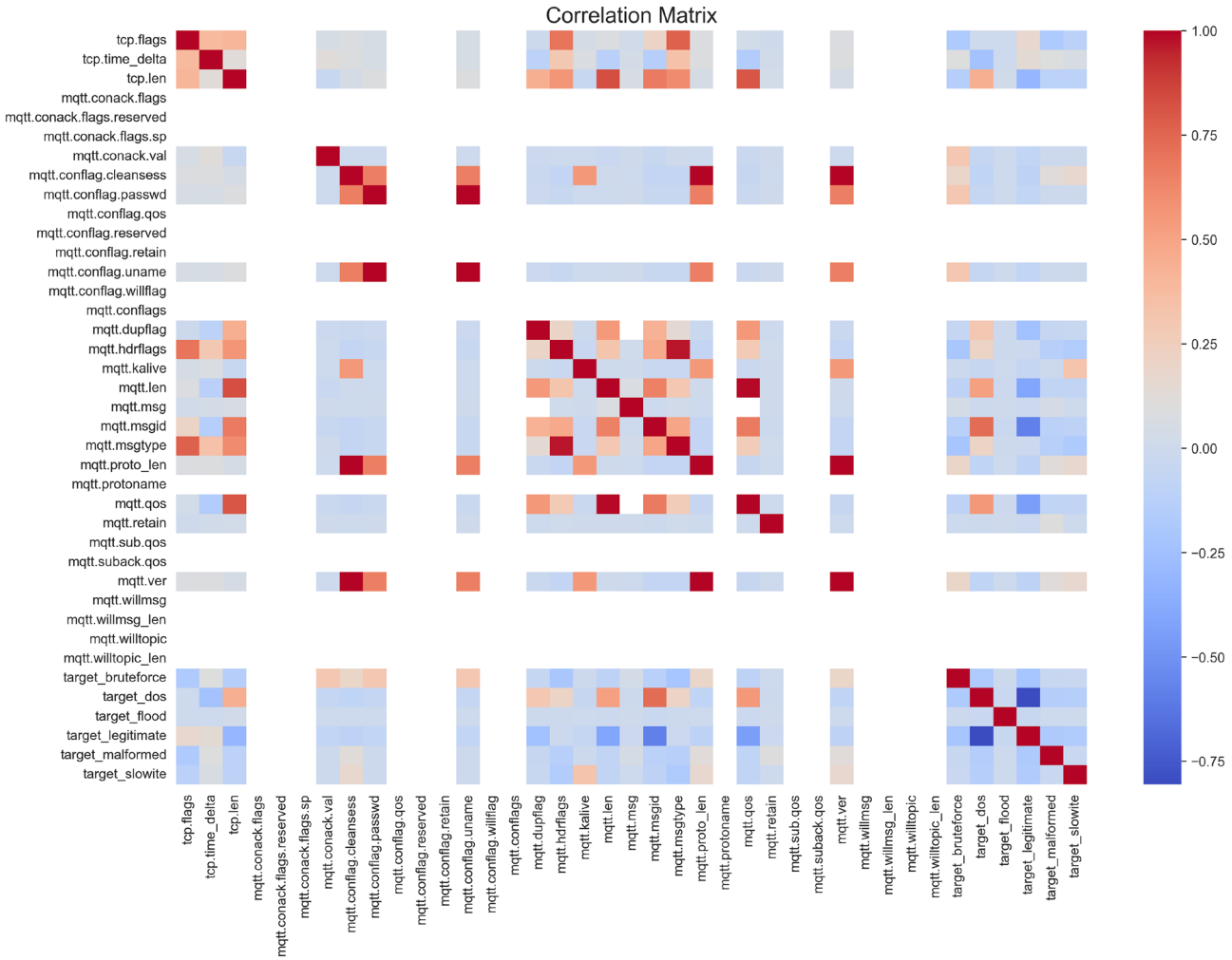


Fig. 12. Correlation matrix among TCP/MQTT features and targets.

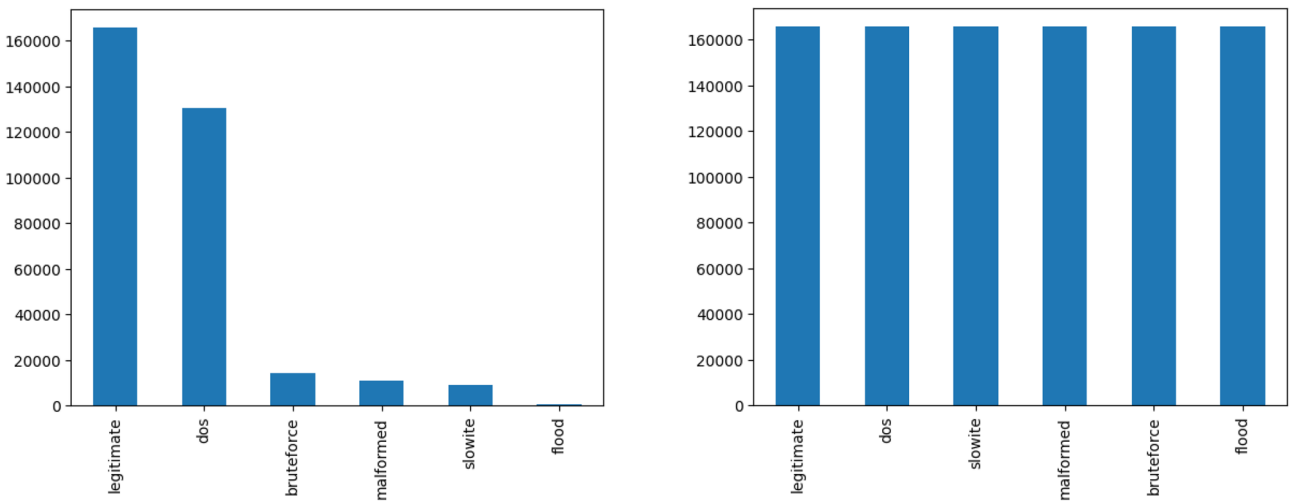


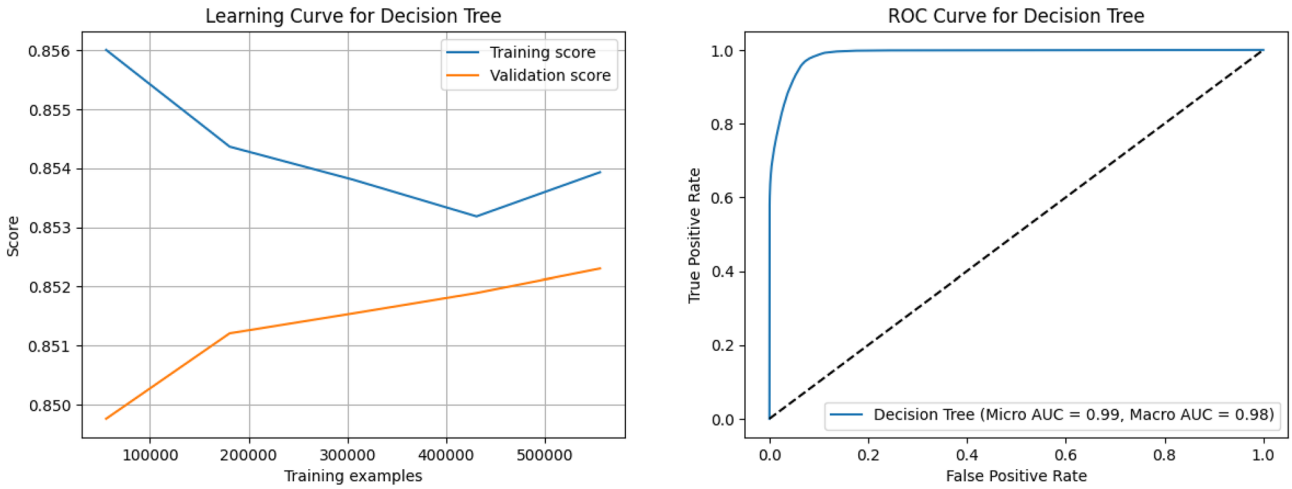
Fig. 13. Class distribution (a) before and (b) after SMOTE resampling.

Model	ROC AUC (Micro)	ROC AUC (Macro)	Accuracy	Precision	Recall	F1-Score	Training Time (s)	Testing Time (s)	Dataset
DT	**0.988**	**0.982**	**0.851**	**0.862**	**0.851**	**0.848**	**1.202**	**0.028**	MQTTset
LightGBM	0.988	0.982	0.848	0.860	0.848	0.844	7.161	0.762	MQTTset
XGB	0.988	0.982	0.848	0.861	0.848	0.843	71.106	0.159	MQTTset
RF	0.988	0.982	0.847	0.858	0.847	0.844	6.249	0.193	MQTTset
GB	0.987	0.982	0.846	0.859	0.847	0.844	534.405	1.617	MQTTset
CatBoost	0.987	0.981	0.842	0.863	0.842	0.835	13.963	0.050	MQTTset
KNN	0.972	0.967	0.825	0.826	0.825	0.825	0.076	3.287	MQTTset
MLP	0.983	0.976	0.824	0.839	0.824	0.839	0.193	0.762	MQTTset
SVM	0.963	0.945	0.782	0.813	0.782	0.813	0.028	0.159	MQTTset
LDA	0.934	0.933	0.667	0.706	0.667	0.663	2.118	0.076	MQTTset
NB	0.790	0.861	0.308	0.602	0.308	0.265	0.307	0.156	MQTTset
AdaB	0.825	0.823	0.306	0.341	0.306	0.289	63.227	4.729	MQTTset

Table 7. Comparison of ML models based on various metrics.

Model	Best Fold	Accuracy	Precision	Recall	F1-Score	AUC (ROC)	Training Time (s)	Testing Time (s)	Dataset
GRU	**Fold 5**	**84.75%**	**0.86**	**0.85**	**0.84**	**0.98**	**646.13**	**8.36**	MQTTset
LSTM	Fold 4	84.63%	0.84	0.84	0.84	0.98	592.04	12.40	MQTTset
RNN	Fold 2	84.28%	0.86	0.84	0.84	0.98	633.34	13.23	MQTTset

Table 8. Performance metrics of DL models on MQTTset dataset.



(a) DT learning curve

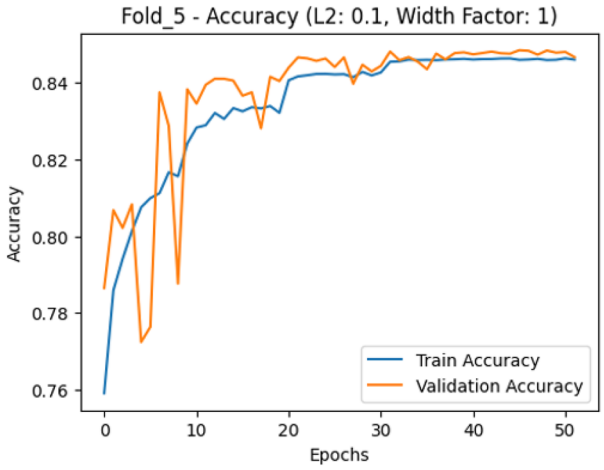
(b) DT ROC curve

Fig. 14. DT diagnostics on MQTTset.

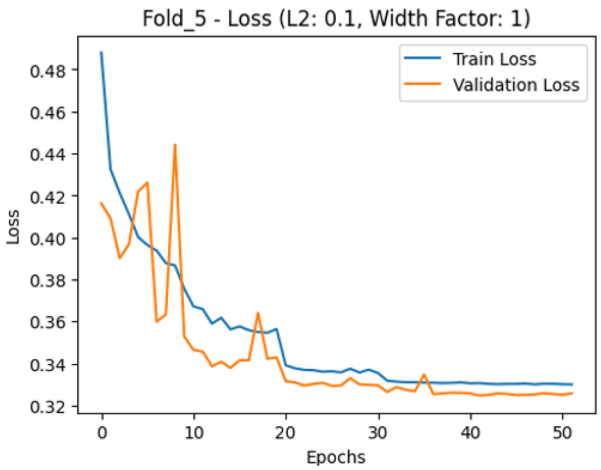
world IoT traffic captured in our deployed testbed. The confusion matrices in Fig. 17 demonstrate a substantial reduction in misclassifications after retraining. As described earlier, the quantitative ratios were computed using the normalized per-class precision, recall, and F1-score formulations defined by Sokolova and Lapalme[28], ensuring cross-model consistency in metric derivation.

In Fig. 17-a, the retrained DT model exhibits strong diagonal dominance across all attack types, confirming robust class separation. The true positive rates exceeded 90% for all classes, with residual confusion limited to 3% between *Legitimate* and *Bruteforce* and 2% between *Malformed* and *Flood*. This reflects that remaining errors originate mainly from overlapping temporal patterns or near-identical payload structures rather than model bias. The distribution of false positives became sparse and class-specific, validating that the retrained DT effectively captured protocol-level semantics.

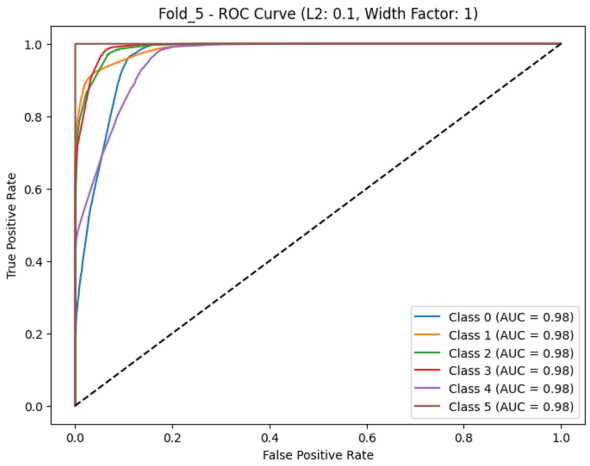
In contrast, the retrained GRU model (Fig. 17-b) achieved even higher discriminative precision, with 95–98% correct classification rates for *DoS*, *Slowite*, and *Malformed* classes. Minor confusion ($\leq$5%) persisted between *Legitimate* and *Bruteforce*, though the sequential architecture yielded superior adaptation to time-correlated



(a) GRU learning—accuracy (Fold 5)

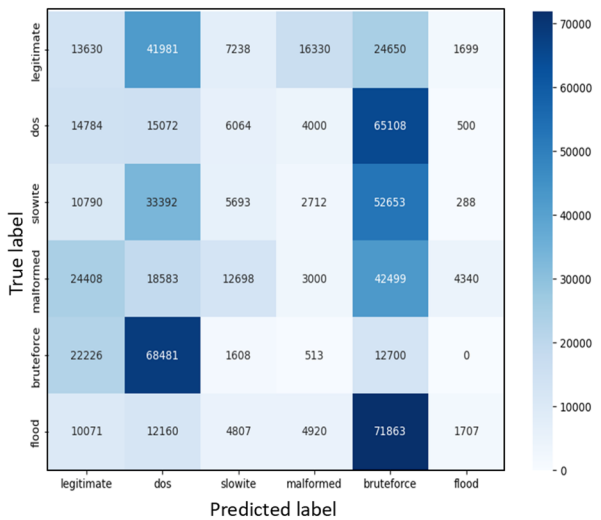


(b) GRU learning—loss (Fold 5)

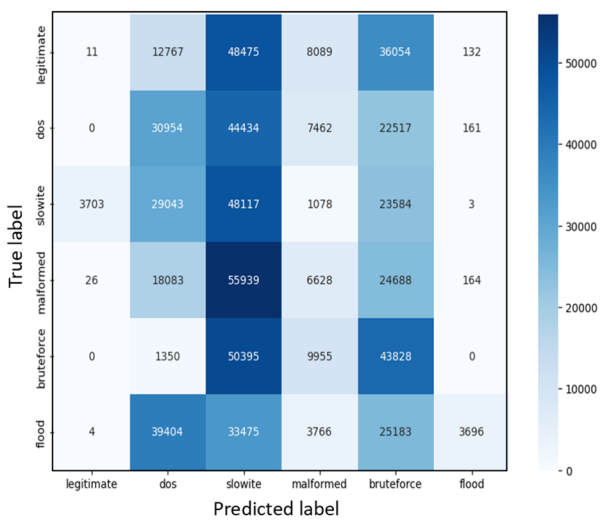


(c) GRU ROC (best fold)

Fig. 15. DL diagnostics on MQTTset for GRU.

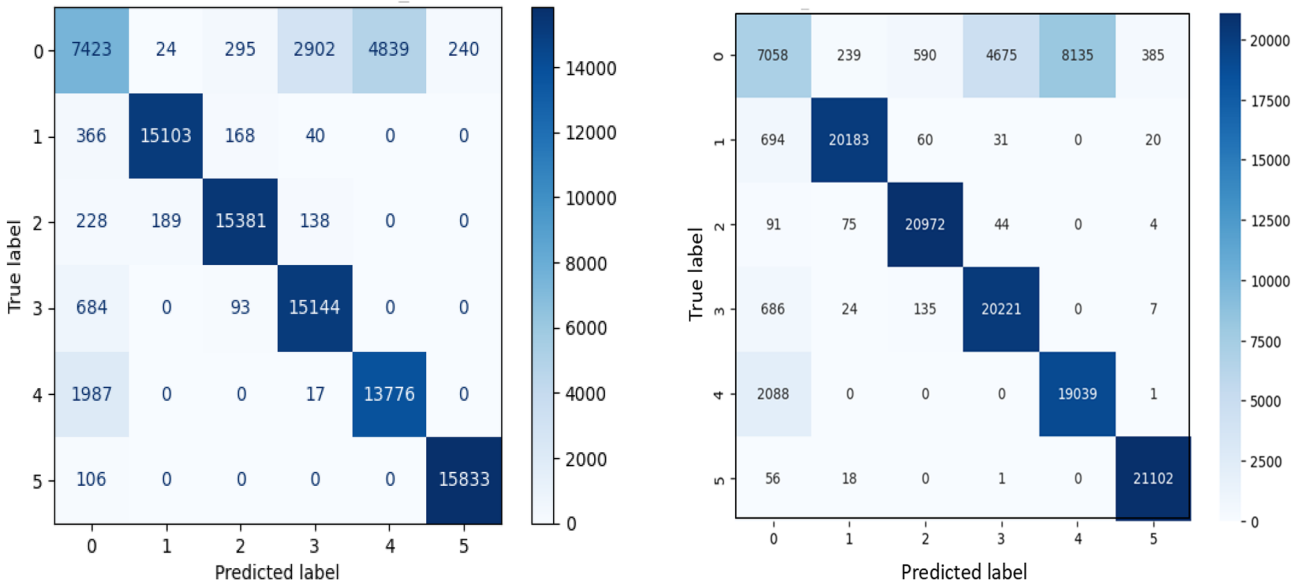


(a) DT



(b) GRU

Fig. 16. Confusion Matrices on MQTTEEB-D for (**a**) DT and (**b**) GRU pre-trained on MQTTset.



(a) Retrained DT model (b) Retrained GRU model

Fig. 17. Confusion matrices of retrained models on MQTTEEB-D dataset.

Model	Dataset	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
DT	Pre-trained on MQTTset, tested on MQTTEEB-D	0.0800	0.1200	0.0800	0.0800
DT	**Retrained on MQTTEEB-D**	**0.8700**	**0.8700**	**0.8700**	**0.8600**
GRU	Pre-trained on MQTTset, tested on MQTTEEB-D	0.2100	0.2900	0.2100	0.1600
GRU	**Retrained on MQTTEEB-D**	**0.8655**	**0.8599**	**0.8655**	**0.8521**

Table 9. Global comparison of pre-trained vs. retrained models.

variations in MQTT message flows. These results indicate that GRU's recurrent structure better handles transient burst patterns and message-rate irregularities, reinforcing its suitability for dynamic IoT traffic.

Quantitative comparisons in Table 9 confirm the consistency of these improvements across all metrics. The retrained DT improved its accuracy from 8% to 87%, while the GRU model's F1-score increased from 16% to 85.21%. Notably, the macro F1-score gain ($\Delta F_1 = +69.21$ pp) aligns with the recall enhancement visible in Fig. 17, verifying stronger sensitivity to minority attack classes and reduced false negatives. These results demonstrate that retraining on realistic domain-specific data corrected the class imbalance and overfitting issues, seen in the pre-trained models (Fig. 16).

Figure 18 visualizes these results, where all post-retraining metrics converge above 85%, confirming model stability and absence of overfitting. The GRU model shows a slightly higher recall than DT, attributed to its recurrent memory capturing inter-packet dependencies, while DT maintains comparable precision due to rule-based partitioning efficiency. These gains collectively validate that real-world retraining substantially improves model robustness and operational reliability.

In summary, both retrained models achieved balanced performance across all evaluation metrics, with markedly reduced cross-class confusion and consistent macro-averaged accuracy. These results empirically confirm that retraining on MQTTEEB-D enables domain adaptation and realistic generalization, effectively resolving the performance degradation observed in pre-trained configurations. This step completes the quantitative validation phase before the subsequent live deployment evaluation phase.

The deployment stage represented the ultimate validation of ISAAF's end-to-end reliability, adaptive intelligence, and operational robustness under real-world IoT conditions. While both GRU and DT models demonstrated strong retraining performance, the DT was adopted for deployment due to its compact structure, rapid execution, and low computational footprint on edge hardware. The full ISAAF architecture comprising the Perception, Network, and Application layers was instantiated within the Node-RED orchestration environment, which acted as the execution backbone of the EEBLab Edge Node testbed. As shown in Fig. 20, data acquisition, intelligent inference, automated mitigation, and visualization were executed as a closed-loop real-time feedback cycle, demonstrating continuous adaptation and self-correcting behavior across all IoT layers. Within the Perception layer, distributed Raspberry Pi 4 units interfaced with MySignals boards to capture heterogeneous physiological and environmental parameters. Node-RED subflows labeled *"In-Field Processing"* and *"Serial*

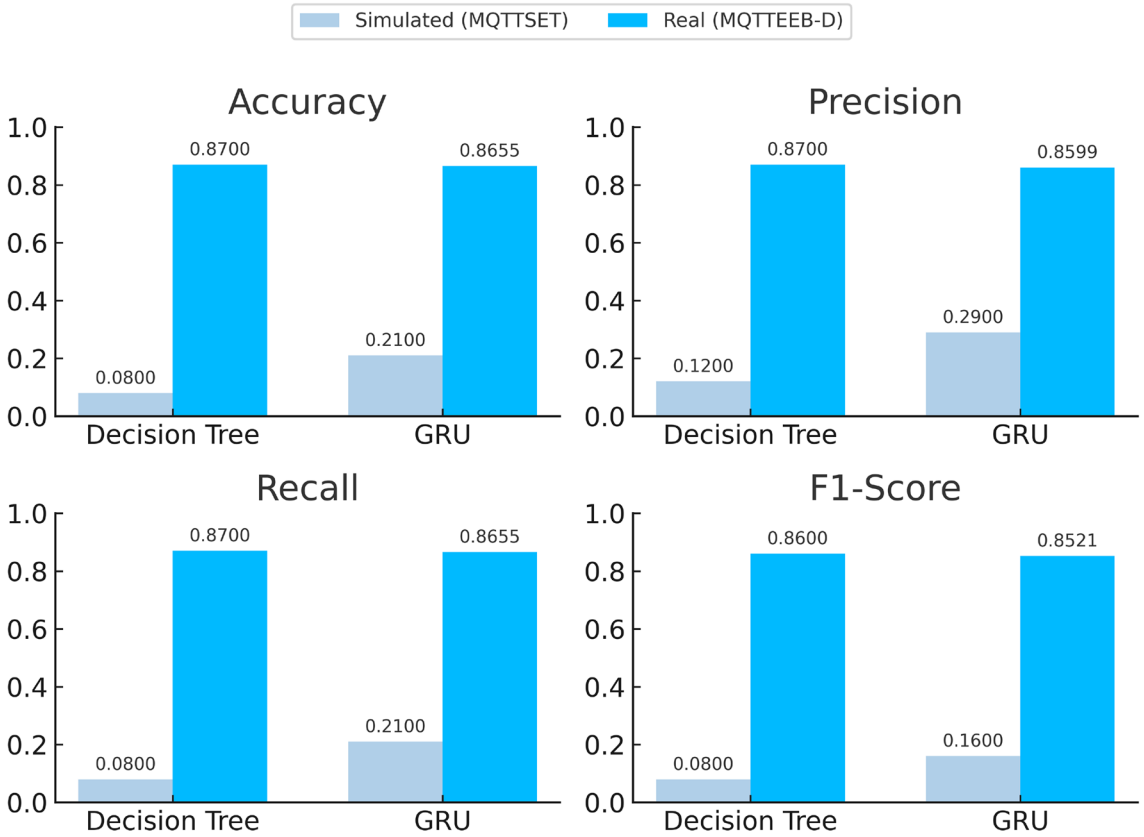


Fig. 18. Performance comparison of pre-trained and retrained models on simulated vs. real-world datasets.

Input" performed on-device parsing, normalization, and timestamping prior to MQTT publication. Debug traces confirmed that every JSON payload was generated by the firmware approximately every 5 s, while end-to-end sensing and publication took 10–11 s, reflecting deterministic acquisition timing and sustained throughput even under continuous sensor operation.

In the Network layer, MQTT packets were routed through the *Traffic Capture* and *AI_Model* pipelines, where they were batched and analyzed in parallel threads by the retrained Decision Tree model. Each inference step executed in under 4 s, while the experimentally measured ∼17 s latency corresponded specifically to end-to-detection time (acquisition → batching → classification), after which mitigation was immediately triggered. REST-based firewall enforcement and dashboard synchronization completed within a 1–3 s, resulting in an effective sensing-to-response latency of ∼17 s across all attack scenarios. System stability under stress was empirically validated using the monitoring interface, shown in Fig. 19. During the execution of a DoS attack, CPU utilization across all four broker cores surged to 60–80 %, and memory exhibited a transient increase, due to accelerated packet buffering, while network ingress traffic spiked above 2 MB/s before rapidly normalizing once the mitigation rule took effect. This resource profile confirms that ISAAF effectively absorbed adversarial bursts while preserving operational continuity and autonomously recovering without manual intervention. Moreover, jitter across consecutive processing batches remained negligible due to the use of parallel-threaded scheduling and stable MQTT timing, ensuring temporal consistency and reliability in the detection output.

At the algorithmic level, ISAAF's DT model leveraged discriminative protocol-level features, inherited from the MQTTEEB-D training process, most notably the `mqtt.qos` attribute. By learning QoS levels 0, 1, and 2 as indicators of delivery reliability, the model was capable of identifying retransmission bursts and malformed negotiation patterns characteristic of *Flood* and *Malformed* attacks directly from live traffic streams. This was performed without requiring any post-processing or external rule sets, demonstrating the intrinsic predictive value of protocol semantics. In the Application layer, once an anomaly threshold was exceeded, the *Mitigation Controller* automatically initiated a REST API request to the local firewall, executed asynchronously in a dedicated thread to ensure uninterrupted handling of Legitimate MQTT sessions. The ThingsBoard interface recorded each detection with timestamp, prediction class, and confidence score, providing a transparent and auditable cybersecurity log. Empirical results confirmed that a *Slowite* attack was detected with 100 % confidence at ∼17 s followed by blocking within 1–3 s; a *Bruteforce* attack was identified with 99.76 % confidence under identical timing conditions; and an *Unknown Attack* was flagged with 90.76 % confidence and preemptively isolated, demonstrating strong generalization beyond signature-based detection.

These results confirm that the complete detection→mitigation→feedback control loop operated within near real-time constraints, with residual latency attributed to Python orchestration and REST communication rather than DT inference, which executed at millisecond scale. The modularity of ISAAF further ensures domain



Fig. 19. System monitoring during a *DoS* attack in MQTTEEB-D showing CPU, memory, and network utilization of the MQTT broker. The results demonstrate real-time detection of system stress and rapid recovery following mitigation.

generalization: the perception layer can seamlessly integrate healthcare, agricultural, or industrial IoT sensors without altering the detection or mitigation logic, validating the framework's adaptability and scalability.

Conclusion and perspectives

This work presented ISAAF, an AI-driven framework for securing MQTT-based IoT systems through predictive analytics and automated prevention. By combining the benchmark MQTTset dataset with the newly developed real-world MQTTEEB-D dataset, we demonstrated the importance of realistic traffic for training robust intrusion detection models. While pre-trained models on MQTTset performed poorly when tested in real-sitting scenarios, retraining DT and GRU on MQTTEEB-D yielded consistent improvements in detection accuracy, confirming the dataset's relevance as a benchmark for IoT security. More precisely, the experimental deployment demonstrated that the ISAAF framework can sustain secure and continuous IoT operation even under adversarial conditions. The layered design linking perception, network, and application components enabled efficient coordination between sensing, analysis, and mitigation while keeping normal MQTT communication active. During live attack scenarios, the prevention mechanism acted precisely by blocking the affected MQTT port, without interrupting Legitimate services, such as dashboard updates, user access, or background telemetry. This targeted action reflects the framework's capacity to isolate threats rather than halt the entire system, allowing administrators to later restore blocked endpoints once verified as safe.

These behaviors confirm that ISAAF is not only accurate in detection but also operationally reliable in maintaining service availability. By integrating lightweight AI inference, real-time orchestration, and adaptive prevention within a closed feedback loop, ISAAF provides a practical and reproducible foundation for intelligent IoT defense. This balance between precision, responsiveness, and continuity demonstrates the framework's potential as a dependable building block for scalable, real-time cybersecurity architectures. In fact, ISAAF demonstrates stable timing, negligible jitter, transparent resource awareness, and autonomous mitigation under diverse attack conditions, confirming its readiness for deployment in large-scale real-world IoT environments.

Future work will focus on reducing latency and enabling lightweight deployment via model compression and hardware–software co-design[29,30]. In parallel, Transformer-based models have shown potential for capturing long-range temporal dependencies in IoT intrusion detection[8,9]. However, their computational complexity limits immediate adoption on constrained devices; we therefore plan to study hybrid variants, suitable for edge integration. Explainable AI (XAI) is another priority for operational trust. Supported by recent explainable MQTT-IDS studies[17,18], we will integrate post-hoc methods, such as SHAP and LIME to make feature contributions interpretable during operation.

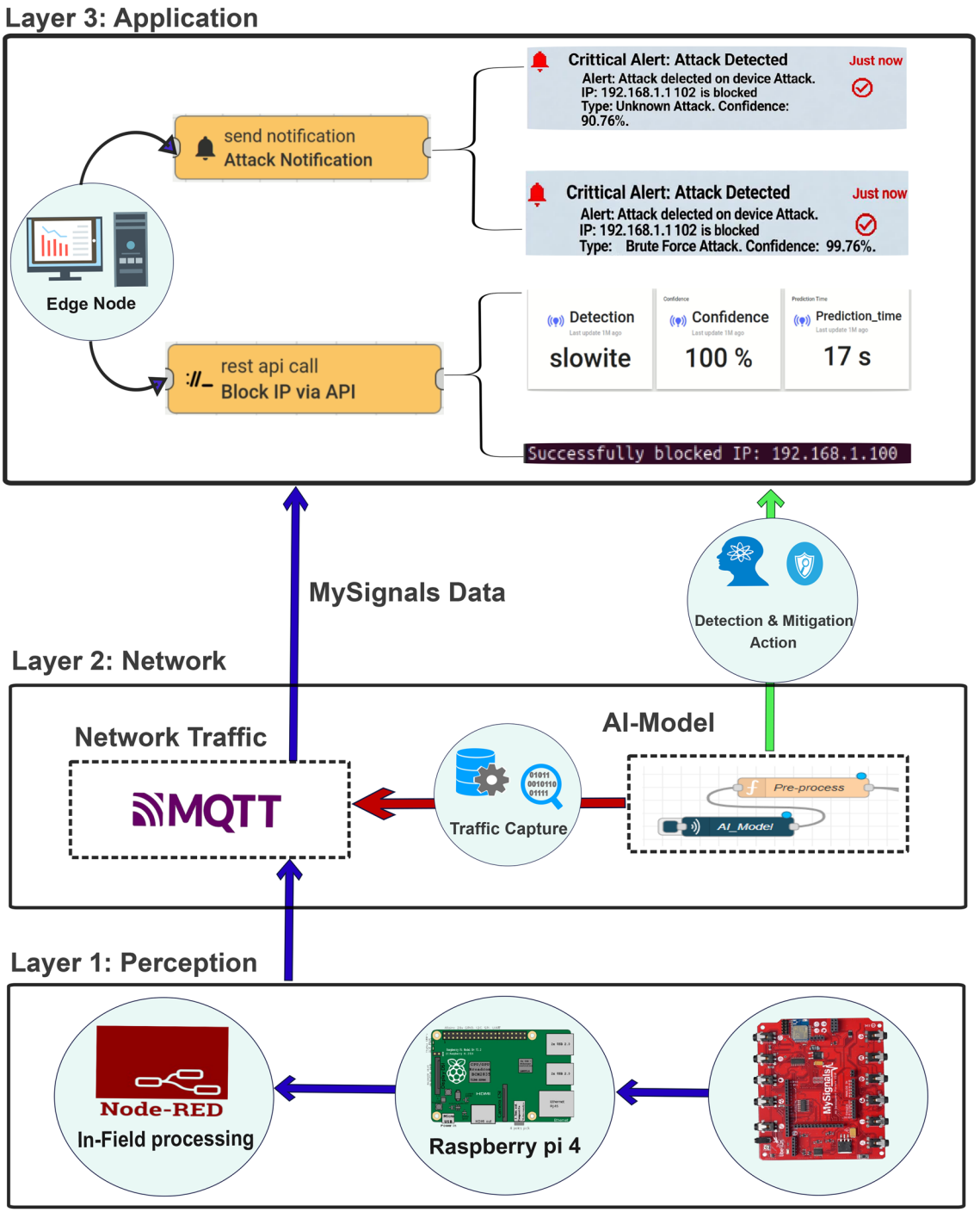


Fig. 20. Operational deployment flow of ISAAF showing the interaction between perception, network, application, orchestration/enforcement, and telemetry/alert layers. The diagram demonstrates how data collected at the edge are processed, classified, and acted upon in real time to ensure adaptive cybersecurity response.

Data availability

Repository name: Mendeley Data. Data identification number: https://doi.org/10.17632/jfttfjn6tr.1. Direct URL to data: https://data.mendeley.com/datasets/jfttfjn6tr/1

Code availability

All source code used for preprocessing, training, and evaluation is openly released: MQTTset Investigation ML/DL Models: https://github.com/Khaoula-Karam/MQTTSET_Investigation_ML_DL_ModelsMQTTEEB-D Retrain Testing Database Creation: https://github.com/Khaoula-Karam/MQTTEEB-D_Model_Retrain_Testin

g_Database_Creation These repositories provide full scripts and pipelines to reproduce the results reported in this paper.

Received: 28 July 2025; Accepted: 11 November 2025
Published online: 29 December 2025

References

1. Malek, Y. N. et al. On the use of iot and big data technologies for real-time monitoring and data processing. *Procedia Comput. Sci.* **113**, 429–434. https://doi.org/10.1016/j.procs.2017.08.281 (2017).
2. Kharbouch, A. et al. Iot and big data technologies for monitoring and processing real-time healthcare data:.. *Int. J. Distributed Syst. Technol.* **10**, 17–30. https://doi.org/10.4018/IJDST.2019100102 (2019).
3. Alotaibi, N. S., Sayed Ahmed, H. I., Kamel, S. O. M. & ElKabbany, G. F. Secure enhancement for mqtt protocol using distributed machine learning framework. *Sensors* **24**, 1638. https://doi.org/10.3390/s24051638 (2024).
4. Kuriakose, N. & Devi, U. *MQTT Attack Detection Using AI and ML Algorithm*, vol. 317, 13–22 (publisherSpringer Nature Singapore, 2022).
5. Vaccari, I., Chiola, G., Aiello, M., Mongelli, M. & Cambiaso, E. Mqttset, a new dataset for machine learning techniques on mqtt. *Sensors* **20**, 6578. https://doi.org/10.3390/s20226578 (2020).
6. Salem, A. H., Azzam, S. M., Emam, O. E. & Abohany, A. A. Advancing cybersecurity: a comprehensive review of ai-driven detection techniques. *J. Big Data* **11**, 105. https://doi.org/10.1186/s40537-024-00957-y (2024).
7. Al Hanif, A. & Ilyas, M. Effective feature engineering framework for securing mqtt protocol in iot environments. *Sensors* **24**, 1782. https://doi.org/10.3390/s24061782 (2024).
8. Hossain, M. S., Rahman, M. M., Islam, M. R. & Karim, M. R. A hybrid cnn–gru-based model for intrusion detection in smart environments using realistic iot traffic. *J. Inf. Priv. Secur.* **20**, 153–172. https://doi.org/10.1080/19393555.2024.2361244 (2024).
9. Wang, Z., Chen, L., Yu, W. & Huang, R. Deep learning-based lightweight ids for iot: Real-time evaluation in live edge environments. *Sci. Reports* **14**, 6789. https://doi.org/10.1038/s41598-024-67890-1 (2024).
10. Liu, C., Zhang, Y., Sun, J., Yang, H. & Wang, Z. Iotgem: A generalization-enhancing methodology for robust iot intrusion detection under domain shift. arXiv preprint arXiv:2401.01343 (2024).
11. Aqachtoul, A. et al. MQTTEEB-D: A Real-World IoT Cybersecurity Dataset for AI-Powered Threat Detection in MQTT Networks. *Data in Brief* **62**, 111897. https://doi.org/10.1016/j.dib.2025.111897 (2025).
12. Rochd, A. et al. Smart microgrids for agriculture: Mg-farm's innovative approach to electrifying farms in north africa-case studies of morocco and algeria. In *booktitle2023 IEEE PES/IAS PowerAfrica*, 1–3 (organizationIEEE, 2023).
13. Aqachtoul, A. et al. Mgfarm: An iot and edge-driven framework for smart and sustainable agricultural practices. In editorMejdoub, Y., editorElamri, A. & editorKardouchi, M. (eds.) *booktitleConnected Objects, Artificial Intelligence, Telecommunications and Electronics Engineering*, 419–425 (publisherSpringer Nature Switzerland, addressCham, 2026).
14. Soni, D. & Makwana, A. A survey on mqtt: A protocol of internet of things (iot). In *booktitleProceedings of the International Conference on Telecommunication, Power Analysis and Computing Techniques (ICTPACT-2017)*, 173–177 (addressChennai, India, 2017).
15. Alzahrani, A. & Aldhyani, T. H. H. Artificial intelligence algorithms for detecting and classifying mqtt protocol internet of things attacks. *Electronics* **11**, 3837. https://doi.org/10.3390/electronics11223837 (2022).
16. Siddharthan, H., Deepa, T. & Chandhar, P. Senmqtt-set: An intelligent intrusion detection in iot-mqtt networks using ensemble multi cascade features. *IEEE Access* **10**, 33095–33110. https://doi.org/10.1109/ACCESS.2022.3161566 (2022).
17. Zeghida, H. et al. Xmid-mqtt: explaining machine learning-based intrusion detection system for mqtt protocol in iot environment. *transport (MQTT)* **4**, 42 (2025).
18. Solanki, M. & Gupta, S. A novel intrusion detection framework using ensemble learning in mqtt iot applications. *Annals Math. Artif. Intell.* 1–23 (2025).
19. Vaccari, I., Aiello, M. & Cambiaso, E. SlowITe, a Novel Denial of Service Attack Affecting MQTT. *Sensors* **20**, 2932. https://doi.org/10.3390/s20102932 (2020).
20. Gao, Z., Cao, J., Wang, W., Zhang, H. & Xu, Z. Online-Semisupervised Neural Anomaly Detector to Identify MQTT-Based Attacks in Real Time. *Secur. Commun. Networks* **1–11**, 2021. https://doi.org/10.1155/2021/4587862 (2021).
21. Alatram, A., Sikos, L. F., Johnstone, M., Szewczyk, P. & Kang, J. J. DoS/DDoS-MQTT-IoT: A dataset for evaluating intrusions in IoT networks using the MQTT protocol. *Comput. Networks* **231**, 109809. https://doi.org/10.1016/j.comnet.2023.109809 (2023).
22. Cabric, M. *Confidentiality, Integrity, and Availability*, 185–200 (publisherElsevier, 2015).
23. Williams, P. & Woodward, A. Cybersecurity vulnerabilities in medical devices: a complex environment and multifaceted problem. *Med. Devices: Evid. Res.* 305. https://doi.org/10.2147/MDER.S50048 (2015).
24. Glaroudis, D., Iossifides, A. & Chatzimisios, P. Survey, comparison and research challenges of iot application protocols for smart farming. *Computer Networks* **168**, 107037. https://doi.org/10.1016/j.comnet.2019.107037 (2020).
25. Paris, I. L. B. M., Habaebi, M. H. & Zyoud, A. M. Implementation of ssl/tls security with mqtt protocol in iot environment. *Wirel. Pers. Commun.* **132**, 163–182. https://doi.org/10.1007/s11277-023-10605-y (2023).
26. Ebrahimpour, E. & Babaie, S. Authentication in internet of things, protocols, attacks, and open issues: a systematic literature review. *Int. J. Inf. Secur.* **23**, 1583–1602. https://doi.org/10.1007/s10207-023-00806-8 (2024).
27. Loshchilov, I. & Hutter, F. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101 (2017).
28. Sokolova, M. & Lapalme, G. A systematic analysis of performance measures for classification tasks. *Inf. processing & management* **45**, 427–437 (2009).
29. Liu, D. et al. A survey of model compression techniques: Past, present, and future. *Frontiers in Robotics and AI* **12**, 1518965 (2025).
30. Liang, T., Glossner, J., Wang, L., Shi, S. & Zhang, X. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* **461**, 370–403 (2021).

Acknowledgements

The authors gratefully acknowledge the support of the MG-FARM project and the broader collaboration provided by the LEAP-RE consortium. We also thank the EEBLab team for their assistance with the IoT testbed deployment and data collection.

Author contributions

K. Karam, A. Aqachtoul, A. Elamrani, M. Najib, N. Rafalia, I. Moumen, J. Abouchabaka, M. Bakhouya: Conceptualization, Methodology, Software Development, Data Curation, Experimentation, Formal Analysis, Investigation, Validation, Results Interpretation, Writing – Original Draft Preparation, Writing – Reviewing and Editing, Supervision, Project Administration, and Funding Acquisition.

Funding
This research is supported by the MGFARM project, which is funded by "Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de l'Innovation, under the LEAP-RE program (Project no: 963530, 2022-2026).

Declarations

Competing interests
The authors declare no competing interests. The corresponding author (K. Karam) confirms this statement on behalf of all authors.

Additional information
Correspondence and requests for materials should be addressed to K.K.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025